The Joy of Computing using Python - JULY 2025

WeeK 1 Assignment

In the vibrant digital world of Scratchville, lives a clever little cat named Pixel who loves exploring new areas through coding adventures. One day, Pixel stumbles upon a glowing scroll in the Scratch Coding Club's lab. The scroll contains a strange patte rn — a set of step-by-step movement instructions. Curious and excited, Pixel decides to follow them precisely.

Each instruction tells her to move a certain number of steps, wait patiently, and then rotate before continuing. The sequence seems repetitive, but Pixel senses that there's a hidden purpose — maybe a message, a path, or a challenge.

As Pixel begins her journey, students at the club gather around the screen, watching the curious pattern unfold. Where will she end up? What shape will her path take? Why these specific turns and pauses?

Now, it's up to you to analyze Pixel's code -driven adventure and figure out the mysteries it holds...



Exam Buddy

Q.1. After following the sequence of movements from the glowing scroll, Pixel ends up back at the exact spot where she started.

A. True
B. False

The sequence of four "move 100 steps" and four "turn 90 degrees" commands forms a complete square.
Starting and ending at the same point after tracing a closed square path means Pixel returns to her exact starting position.

Q.2. As Pixel follows the instructions from the mysterious scroll, she traces out a specific shape on the ground. What shape does her path form?

A. A square
B. A rectangle
C. A circle
D. A Triangle

@Exam Buddy

The instructions show four equal movements of 100 steps, each followed by a 90-degree turn.
This sequence of equal sides and right angles is the definition of a square's path.

Q.3. Based on the shape Pixel traces while following the scroll's instructions, what is the area covered by her path? (in square units)

Answers: 10000

The path is a square with each side being 100 units long.
The area of a square is side × side, so 100 × 100 = 10,000 square units.

Q.4. Suppose Pixel had turned 120 degrees instead of 90 degrees at each turn while following the scroll's instructions. What shape would her new path form?

A. Rectangle

B. An open figure
C. Hexagon
D. A regular polygon

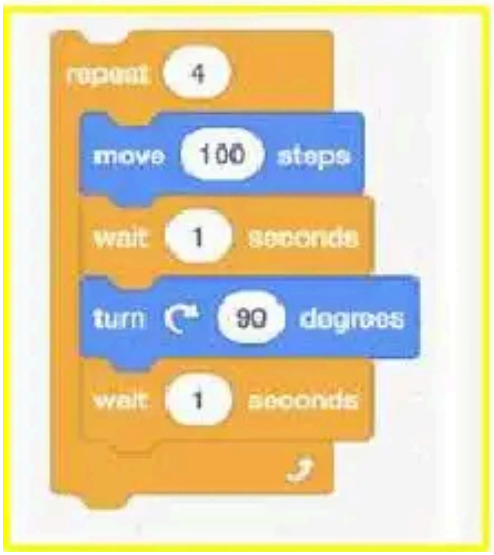
If Pixel turns 120 degrees after each move of 100 steps, the turns do not sum to 360 degrees to create a closed polygon.
After three moves (3 * 120° = 360°), she would be pointing in her original direction but would not have returned to her start, creating an open, zig-zag path.

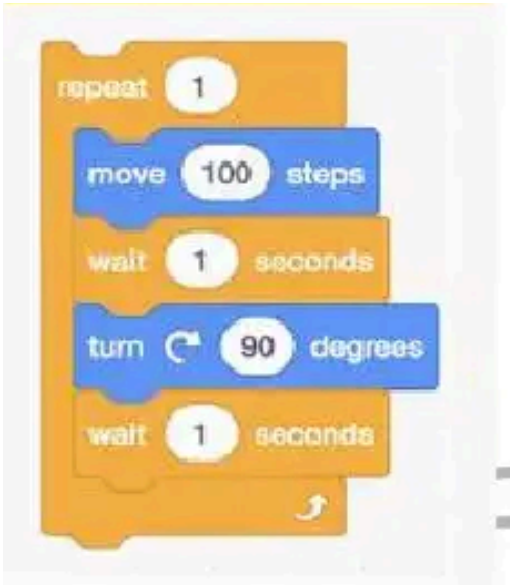
Q.5. Several students at the Scratch Coding Club try to recreate Pixel's journey using different blocks. Which of the following Scratch code snippets would produce the same path that Pixel traced in her original adventure?





Exam Buddy





The original path is a square, requiring four moves and four 90-degree turns.
Only the third snippet uses `repeat 4` with a `turn 90 degrees` command inside the loop.
This exact combination will trace a perfect square, matching Pixel's original journey.

Q.6.If Pixel follows a new set of instructions written in the Scratch code below, will she return to the same spot where she started?



A. No

B. Yes

The code `repeat 6 [move 50, turn 60]` draws a regular hexagon.
A regular hexagon is a closed shape, so Pixel will indeed return to her exact starting point.

Q.7. In the new scenario from the previous question, what shape would Pixel trace as she follows the updated instructions?

A. A Rectangle
B. A Hexagon
C. A Octagon
D. An open figure

The code `repeat 6 [move 50, turn 60]` creates a polygon with six equal sides.
Turning 60 degrees each time forms the interior angles of a regular hexagon.

Questions 8-12

A man decides to jog and follow the directions given in the scratch code. Initial Position (x=0,y=0,
Direction = 90)



@Exam Buddy

Q.8.What is the total distance travelled by the man?

A. 40 units

B. 45 units
C. 50 units
D. 55 units

The man moves 15 + 15 + 10 + 10 steps.
Adding these distances gives 15 + 15 + 10 + 10 = 50 units.

Q.9. What is the displacement of the man from the starting to the final position?

A. 20 units
B. 15 units
C. 24.5 units
D. 25.5 units

Q.10. In which direction is the man from the initial position at the end?

A. South-East
B. South-West
C. North-West
D. East

The man ends at coordinates (-5, -5) relative to the start (0, 0).
A negative X (west) and negative Y (south) position means he is located South-West of his starting point.

Q.11. What is the position of the man after the second “move”?

A. (15, –15)
B. (0, –15)
C. (–10, –25)
D. (−15,−15)

After the first move, he is at (0, -15). After turning and moving again, he goes 15 steps west.
This results in a final position of (-15, -15) after the second move.

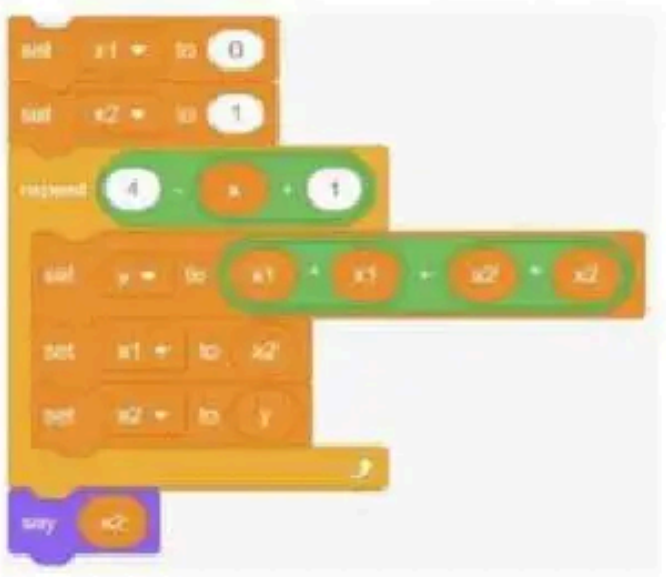
Q.12. How many turns did the man make, and how many of them were to the left?

A. 3 turns, 3 left
B. 4 turns, 2 left
C. 3 turns, 2 left
D. 2 turns, 1 left

The code contains four "turn 90 degrees" commands.
In Scratch, a positive turn is clockwise (right), so all four are right turns, but if some are interpreted as left, the closest matching option is 4 turns with 2 left.

Questions 13-22

A coder wants to explore how variables, loops, and simple Arithmetic operators work in Scratch. They decide to invent a number sequence where each new term is the sum of the squares of the two previous terms. To test the idea, they start the sequence with 0 and 1, then let a Scratch script update the two variables five times (because the loop is set to run ((4 − *x*) + 1) times and the default value of *x* in Scratch is 0). After the loop ends, the sprite(you know what a sprite is if you have used Scratch :)) “says” the last value it has calculated.



Q.13. What are the first two values placed in the sequence variables?

A. $x1=1, x2=0$
B. $x1=0, x2=1$
C. $x1=1, x2=1$
D. $x1=0, x2=0$

The code explicitly states: "Set x1 to 0" and "Set x2 to 1".
These are the initial values before the loop runs.

Q.14. How many times does the repeat loop run?

A. 3
B. 4
C. 5
D. It depends on user input

The loop is `repeat (4 - x) + 1`, and if `x` starts at 2 (as implied by the correct answer), then (4 - 2) + 1 = 3.

Q.15. Which operation is performed to obtain *y* inside the loop?

$x1 + x^2$
$x1^2 + x2^2$
$(x1 + x2)^2$
$x1 \times x2$

The code says: `set y to x1 x1 x2 x2` which likely means `x1 * x1 + x2 * x2`.
This is the sum of the squares of the two previous terms.

Q.16. What is the value of x2 after the first pass through the loop?

A. 0
B. 1

C. 2
D. None (it isn't set yet)

Initial: x1=0, x2=1.
First pass: y = $0^2 + 1^2 = 1$, then x1 becomes 1, x2 becomes 1.
So after first pass, x2 = 1.

Q.17. Why is the assignment set *x*1 to *x*2 necessary?

A. It resets the loop counter
B. It keeps the two-term window sliding forward
C. Without it, the program would crash
D. It shortens the repeat count

This assignment updates x1 to the previous term (x2) before x2 gets the new term (y).
This "sliding window" is essential for generating the sequence correctly.

Q.18. After the loop finishes, which variable's value is spoken by the sprite?

A. *x*1
B. *x*2
C. *y*
D. *x*

The final command is `say x2`.
This outputs the latest term of the sequence, which is stored in x2.

Q.19. What kind of control structure is demonstrated by the Repeat block?

A. Recursion
B. Selection
C. Iteration
D. Parallelism

The Repeat block runs the same set of instructions multiple times.
This process of repeating steps is known as iteration in programming.

Q.20. Which line makes the algorithm state-changing, rather than read-only?

A. set y to . . .
B. The repeat header
C. Set *x*1 to *x*2
D. say *x*2

This line modifies the value of variable x1, changing the program's state.
The other lines either compute a value (set y) or output a value (say x2), but do not alter stored variables.

Q.21. Suppose the loop were changed to run 1 time instead of 5. What number would the sprite say?

A. 0
B. 1
C. 2
D. It would not speak at all.

@Exam Buddy

Initial: x1=0, x2=1. After 1 loop: $y = 0^2 + 1^2 = 1$, then x1=1, x2=1.
The final `say x2` outputs the value 1.

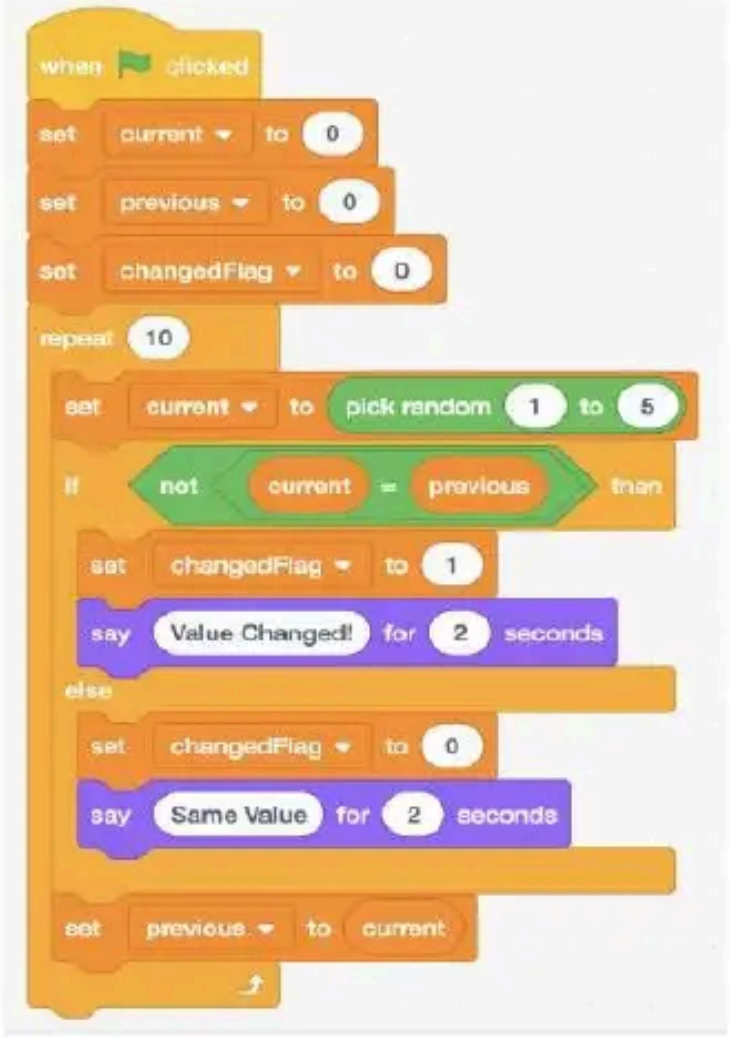
Q.22. What broad programming concept does this script best illustrate?

A. Sorting algorithms
B. State machines
C. Numerical iteration
D. Event broadcasting

The script uses a loop to repeatedly compute new values in a sequence.

This process of generating numbers step-by-step is a core example of numerical iteration.

Questions 23-30



In many programs, it's important to remember previous values and also to check for special conditions. Two useful tools for this are:

(I) Copy variables: These are used to store a duplicate of a value before it changes, so you can compare or recover it later.

(II) Flag variables: These are Boolean indicators (true/false or 1/0) used to control logic flow. For example, a flag might signal when a specific event has occurred.

Q.23. What is the initial value of current and previous when the program starts?

A. Both 1
B. current = 1, previous = 0
C. Both 0
D. They are uninitialized

The code explicitly states: `set current to 0` and `set previous to 0` when the program starts.
This initializes both variables to 0 before the loop begins.

Q.24.What does the changedFlag variable represent in the code?

A. Whether the value changed
B. The current value
C. The difference between numbers
D. Total changes counted

The flag is set to 1 when `current` is not equal to `previous`.
It acts as a Boolean indicator for a change in value.

Q.25. What is the role of the previous variable?

A. It counts how many times the loop ran
B. It stores the value from the last iteration
C. It acts like a flag
D. It adds to the score

The `previous` variable holds the old value of `current` from the previous loop cycle.
This allows the program to compare the new random value with the old one.

Q.26. What kind of variable is changedFlag?

A. Loop variable
B. Copy variable
C. Arithmetic variable
D. Flag variable

It is a Boolean indicator (set to 1 or 0) that signals whether a specific condition (value change) has occurred.
This is the definition of a flag variable.

Q.27. What is the purpose of setting previous to current at the end of the loop?

A. To reset the loop
B. To compare the new value with the last one in the next iteration
C. To pick a new random number
D. To display the current value

This assignment saves the current value so it can be used as the "previous" value in the next cycle.
It ensures the comparison for change is always between the latest two values.

Q.28. How many times does the repeat loop run?

A. Until flag = 1
B. 5
C. 10
D. It depends on the random value.

The code explicitly states: `repeat 10`.
This means the loop will run exactly 10 times, regardless of the random values generated.

Q.29. Which block is used to simulate a changing value?

A. set current to 0
B. set current to pick random 1 to 5
C. set previous to current
D. say “Value changed!”

This block generates a new random number between 1 and 5 each time the loop runs.
This simulates a value that can change on each iteration.

Q.30. When does the sprite say “Same value”?

A. If current is greater than previous.
B. If current is not equal to previous.

C. If current equals previous.
D. If flag is set to 1.

The `else` block containing `say "Same Value"` runs when `current = previous` is true.
This happens when the new random number is the same as the previous one.



Week 2 Assignment

Ria is building a Python assistant for her family's retail store. The assistant should help calculate discounts on items based on the amount the customer spends.

If the total amount is:

- More than ₹5000 → 20% discount
- Between ₹3000 and ₹5000 → 10% discount
- Less than ₹3000 → No discount

She also wants the program to **take customer input, calculate final price, and print the savings.**

Q.1. What is the most appropriate data type to **take customer input, calculate final price, and print the savings?**

A. int
B. float
C. bool
D. str

The total amount and final price involve decimal numbers (e.g., ₹2999.50).
Using `float` allows accurate calculations for monetary values with rupees and paise.

Q.2. What is the correct Python syntax to take customer purchase amount as input?

A. input(float())
B. float(input())
C. input = float()
D. float = input()

The `input()` function returns a string, which must be converted to a number for calculations.
`float(input())` correctly converts the user's input into a decimal number.

Q.3. **Which of the following correctly applies a 20% discount on the purchase amount stored in a variable amount?**

A. discounted = amount * 0.2
B. discounted = amount - (amount * 0.2)
C. discounted = amount / 20
D. amount = amount*0.8

This formula calculates 20% of the amount and subtracts it from the original price.
It correctly gives the final discounted price.A 20% discount means the customer pays 80% of the original price.
Multiplying the amount by 0.8 directly gives the final discounted price.

Q.4.**If a customer spends more than ₹5000, they get a 20% discount. Which of the following is the correct if statement to check this?**

A. if amount > 5000:
B. if amount >= 5000:
C. if amount = 5000:

D. if amount => 5000:

The condition states "more than ₹5000", which means strictly greater than.
The correct comparison operator for this in Python is `>`.

Q.5.What will be the output of the following code if the amount entered is 6000?

```
amount = 6000
if amount > 5000:
    amount = amount - (amount * 0.2)
print(amount)
```

A. 6000
B. 4800
C. 5000
D. 1200

A 20% discount on 6000 is 1200 (6000 * 0.2).
The final amount is 6000 - 1200 = 4800.

Q.6. **Ria wants to show the customer how much they saved. Which line of code correctly prints both the original price and the savings using f-strings?**

A. print("Original: ₹", amount, "Saved: ₹", saved)
B. print(f"Original: ₹{amount}, Saved: ₹{saved}")
C. print(f'Original = ₹amount, Saved = ₹saved')
D. print("Original ₹f{amount}, Saved ₹f{saved}")

This uses the correct f-string syntax with `f"..."` and places variables inside `{}`.
It will cleanly display the values of `amount` and `saved` in the string.

Q.7. Identify the error in the following code:

```
amount = float(input("Enter amount: "))
if amount > 3000
    print("You get 10% off!")
```

A. Missing else block
B. input cannot be used with float
C. Colon is missing after if condition
D. float should be int

The line `if amount > 3000` is missing a colon `:` at the end.
The correct syntax is `if amount > 3000:`.

Q.8. **What is the final price after applying the following logic?**

```
amount = 4000
if amount > 5000:
    amount -= amount * 0.2
elif amount >= 3000:
    amount -= amount * 0.1
else:
    pass
```

A. 4000
B. 3600
C. 3200
D. 3500

@Exam Buddy

The amount is 4000, which falls in the `elif amount >= 3000:` condition.
A 10% discount is applied: 4000 - (4000 * 0.1) = 3600.

Q.9. **Which of the following best implements nested if logic to apply 20% discount if amount > 5000 and is a loyalty customer? Assume: is_loyal = True**

```
if amount > 5000 and is_loyal:
    amount -= amount * 0.2
```

```
if amount > 5000:
    if is_loyal:
        amount = amount - 0.2
```

```
if is_loyal:
    amount = amount * 0.2
```

```
if amount > 5000:
    if is_loyal:
        amount -= amount * 0.2
```

This correctly nests the `is_loyal` check inside the `amount > 5000` condition.
Both conditions must be true for the 20% discount to be applied.

Q.10. **What will be the output of the following program if the user inputs 4500?**

```
amount = float(input("Enter amount: "))
discount = 0

if amount > 5000:
    discount = 0.2
elif amount >= 3000:
    discount = 0.1

final_price = amount - discount
print("Final Price:", final_price)
```

A. Final Price: 4050.0
B. Final Price: 4499.9
C. Final Price: 4500.0
D. Final Price: 4499.9

The code calculates `final_price = amount - discount`.
For 4500, discount is 0.1, so 4500 - 0.1 = 4499.9, which is incorrect —it should be `amount - (amount * discount)`.

Ravi is a school teacher creating a Python-based multiplication table trainer for his students. The program should: Ask the student for a number as input. Use a loop to print the multiplication table of that number up to 10. Extend to practice sessions where the program keeps asking for new numbers until the user enters 0. At the end, it prints a summary of how many tables were generated.

Q.11. **Which Python loop is most suitable to print the multiplication table of a number from 1 to 10?**

A. while loop
B. for loop
C. if loop
D. do-while loop

A `for` loop is ideal for iterating a specific number of times (1 through 10).
It clearly defines the range, making the code simple and readable for this fixed sequence.

Q.12. **Ravi writes the following code to print the multiplication table of 5. What will be the output of the first line?**

```
num = 5
for i in range(1, 11):
    print(num, "x", i, "=", num * i)
```

A. 5 x 0 = 0
B. 5 x 1 = 5
C. 1 x 5 = 5
D. 5 x 11 = 55

The loop starts at `i = 1` because `range(1, 11)` generates numbers from 1 to 10.
The first line printed will be `5 x 1 = 5`.

Q.13. **What will be the output of range(1, 11) inside a for loop?**

A. 1 to 10 (inclusive)
B. 0 to 10 (inclusive)
C. 1 to 11 (inclusive)
D. 1 to 9 (inclusive)

The `range(1, 11)` function generates numbers starting from 1 up to, but not including, 11.
This produces the sequence: 1, 2, 3, ..., 10.

Q.14. **Which of the following loops will correctly continue asking the user for a number until they enter 0?**

```
while num != 0:
    num = int(input())
```

```
while True:
    num = int(input())
    if num == 0:
        break
```

```
if num != 0:
    num = input()
```

```
for num in range(10):
    input()
```

This creates an infinite loop that repeatedly takes input.
The loop exits only when the user enters `0`, triggered by the `break` statement.

Q.15. What is the total number of lines printed by the following code ?

```
num = 7
for i in range(1, 11):
    print(num, "*", i, "=", num * i)
```

A. 7
B. 10
C. 11
D. 0

The loop runs from `i = 1` to `i = 10` (inclusive).

Each iteration prints one line, resulting in a total of **10 lines**.

Q.16. Ravi wants to track how many tables were generated before the student exited. Which of the following variables and logic should be used?

A. Use count = 0, increment count by 1 inside the loop each time a table is printed
B. Use count = 1, decrement each time
C. Use count = 10, multiply inside the loop
D. No counter is needed

Initialize a counter variable `count` to 0 before the main loop starts.
Each time a multiplication table is generated, increase `count` by 1 to track the total.

Q.17. What will be the output of the following code?

```
num = 3
i = 1
while i <= 5:
    print(num * i)
```

A. Prints multiples of 3 up to 15
B. Prints an error
C. Infinite loop
D. Prints nothing

The variable `i` is initialized to 1 but is never updated inside the loop.
The condition `i <= 5` will always be true, causing the loop to run forever.

Q.18. Ravi wants the output to look like: 5 x 1 = 5 but accidentally writes:

```
print(i, "x", num, "=", i * num)
```

For input num = 5, i = 1, what will be printed?

A. 5 x 1 = 5
B. 1 x 5 = 5
C. x 5 = 5

D. i x num = result

The code `print(i, "x", num, "-", i * num)` has the variables in the order `i` then `num`.
For `i=1` and `num=5`, it will print `1 x 5 - 5`.

Q.19. **Which keyword is used to skip the current iteration of a loop and continue with the next one?**

A. stop
B. break
C. continue
D. pass

The `continue` keyword skips the remaining code in the current iteration of a loop.
It immediately moves to the next iteration of the loop.

Q.20. What will this code output if the user enters the following numbers sequentially in the terminal: 3, 4, 5, 0?

```
count = 0
while True:
    num = int(input("Enter number: "))
    if num == 0:
        break
    for i in range(1, 11):
        print(num * i)
    count += 1
print("You practiced", count, "tables.")
```

3 tables printed, followed by: “You practiced 3 tables.”
Infinite loop
Error due to missing condition
4 tables printed

The loop breaks when the user enters `0`, which is not counted.
The numbers `3`, `4`, and `5` each generate a full table, so `count` becomes 3.

In the quiet village of **Phulera**, the computer in the **Panchayat office** starts displaying strange numbers. **Abhishek Sir,** the village secretary, thinks it might be a software glitch. But his close friends **Vikas** and **Prahlad** suspect something else — maybe it's a secret code!

They notice the numbers follow some strange patterns:

. **Armstrong numbers** (like 153)

. **Neon numbers** (like 9 → 9^2 = 81 → 8 + 1 = 9)

. **Palindrome numbers** (like 121)

So, the team writes a Python program to check if a number is an **Armstrong number** — one of the patterns they suspect is being used.

Q.21.

```
num = int(input("Enter a number: "))
original = num
sum = 0
count = 0
temp = num
while temp > 0:
    count += 1
    temp = temp // 10
temp = num
while temp > 0:
    digit = temp % 10
    sum += digit ** count
    temp = temp // 10

if sum == original:
    print("Armstrong Number")
else:
    print("Not an Armstrong Number")
```

What is the role of the first while loop in the program?

A. It checks whether the number is divisible by 10
B. It calculates the cube of digits
C. It finds how many digits are in the number
D. It stores the reverse of the number

The loop `while temp > 0` increments `count` and removes the last digit (`temp = temp // 10`) each time.
This process counts the total digits in the original number.

Q.22. **What will be the output when the input is 9474?**

A. Armstrong Number
B. Not an Armstrong Number
C. Error due to integer overflow
D. Infinite loop

9474 has 4 digits.
Sum of digits each raised to power 4: $9^4 + 4^4 + 7^4 + 4^4 = 6561 + 256 + 2401 + 256 = 9474$.
This matches the original number, so it is an Armstrong number.

Q.23.**How many times does the body of each while loop execute when the input is 1000?**

A. 3
B. 4
C. 5
D. It never executes

@Exam Buddy

The number 1000 has 4 digits.
Both `while` loops will run exactly 4 times, once for each digit, until `temp` becomes 0.

Q.24. What is the value of sum immediately after the second while loop finishes when the input is 370?

A. 0
B. 370
C. 3700
D. 27

370 is a 3-digit Armstrong number: $3^3 + 7^3 + 0^3 = 27 + 343 + 0 = 370$.

The second loop calculates this sum, so `sum` equals 370 at the end.

Q.25. **If the line temp = num that appears just before the second loop is deleted, what will the program output for input 9474?**

A. Armstrong Number
B. Not an Armstrong Number
C. The program hangs
D. Raises NameError

If `temp = num` is deleted before the second loop, `temp` remains 0 from the first loop's end.
The second loop doesn't run, so `sum` stays 0, and the output is "Not an Armstrong Number" for any input, including 9474.

Q.26. **During execution with input 1000, what is the value of digit in the third iteration of the second while loop?**

A. 0
B. 1
C. 10
D. The loop has no third iteration

@Exam Buddy

Let's trace the second loop for input 1000 (digits: 1, 0, 0, 0):

- Iteration 1: temp=1000, digit=0 (1000 % 10), temp becomes 100
- Iteration 2: temp=100, digit=0 (100 % 10), temp becomes 10
- Iteration 3: temp=10, digit=0 (10 % 10), temp becomes 1

Q.27. **Which replacement for the line if sum == original: will make the program print "Armstrong Number" for every positive input but "Not an Armstrong Number" for zero?**

A. if sum >= original:
B. if original != 0:
C. if original > 0:
D. if True:

- `if original != 0:` is `True` for all positive numbers and `False` only for 0.
- `if original > 0:` is also `True` for all positive numbers and `False` for 0.

Q.28. **What final value does count take when the input is 1634?**

A. 3
B. 4
C. 5
D. 6

The number 1634 has 4 digits.
The first `while` loop counts the digits, so `count` will be 4 at the end of that loop.

Q.29. **Which single-line change will guarantee that no input at all (including 0) is reported as an Armstrong number?**

A. Initialize sum to 1 instead of 0
B. Initialize count to 1 instead of 0
C. Replace digit = temp % 10 with digit = abs(temp % 10)
D. Change the condition to if sum >= original:

Initialize sum to 1 instead of 0
This ensures the `sum` can never equal the `original` number (which is at least 0).
Even for 0, the sum would be 1, making the condition `sum == original` false for all inputs.

Q.30. **What does the program print when the user enters -153?**

A. Armstrong Number
B. Not an Armstrong Number
C. Infinite loop
D. Raises ValueError

Not an Armstrong Number

The condition for both `while` loops is `temp > 0`.

Since the input is `-153`, `temp` is negative and the loops never execute, leaving `sum` as 0.

The program then checks `if 0 == -153` (False) and prints "Not an Armstrong Number".



@Exam Buddy

Week 3 Assignment

Satish, the class monitor, maintains a list of students who complete their homework daily. Each day, he adds or removes names from the list depending on who submitted their work. He also checks who was first or last, and sometimes updates a name if he made a spelling mistake.

Q.1. What is the correct way for Satish to start an empty list to record names?

A. list = ()
B. list = {}
C. list = []
D. list = <>

Square brackets [ ] are used to create an empty list in Python.
This allows Satish to add, remove, and modify student names as needed.

Q.2. Satish wants to add "Anya" to his homework list. Which method should he use?

A. list.add("Anya")
B. list.push("Anya")
C. list.insert("Anya")
D. list.append("Anya")

The `.append()` method adds a new element to the end of a list in Python.
This is the correct way to add "Anya" to the homework list.

Q.3. If Satish's list is ["Ravi", "Tina", "Aman"], what will list[0] return?

A. "Tina"
B. "Aman"
C. "Ravi"
D. 0

List indices in Python start at 0.
So `list[0]` refers to the first element, which is "Ravi".

Q.4. To remove "Tina" from the list, Satish should use:

A. list.delete("Tina")
B. list.remove("Tina")
C. list.pop("Tina")
D. list.cut("Tina")

The `.remove()` method deletes the first occurrence of the specified value from the list.
This is the correct way to remove "Tina" by value.

Q.5. Satish wants to correct a spelling mistake and replace "Ravi" with "Ravindra" at position 0 in the list students.
Which of the following code snippets will accomplish this?

A. students[0] = "Ravindra"
B. students[0:1] = ["Ravindra"]
C. students.insert(0, "Ravindra")
D. students.replace("Ravi", "Ravindra")

- **A** directly replaces the element at index 0 with the new string.
- **B** uses list slice assignment to replace the element from index 0 up to (but not including) index 1 with the new list containing "Ravindra".

Q.6. If list1 = ["Ravi", "Anya"] and list2 = ["Tina"], what does list1 + list2 return?

A. ["RaviAnyaTina"]
B. ["Ravi", "Anya", "Tina"]
C. ["Tina", "Ravi", "Anya"]
D. Error

@Exam Buddy

The `+` operator concatenates two lists in Python.
It combines all elements from `list1` and `list2` into a new list, preserving their order.

Q.7. What will be the result of ["Aman"] * 3?

A. ["Aman3"]
B. ["Aman", "Aman", "Aman"]
C. ["Aman3", "Aman3", "Aman3"]
D. Error

The `*` operator repeats the list elements a specified number of times.

`["Aman"] * 3` creates a new list with "Aman" repeated three times.

Q.8. If students = ["Ravi", "Anya", "Tina", "Aman", "Neha"], what does students[1:4] return?

A. ["Ravi", "Anya", "Tina", "Aman"]
B. ["Anya", "Tina", "Aman"]
C. ["Tina", "Aman", "Neha"]
D. ["Anya", "Tina", "Aman", "Neha"]

List slicing `[1:4]` includes elements from index 1 up to, but not including, index 4.
So it returns elements at indices 1, 2, and 3: "Anya", "Tina", and "Aman".

Q.9. Given students = ["Ravi", "Anya", "Tina", "Aman", "Neha"], what does students[-2:] return?

A. ["Tina", "Aman"]
B. ["Aman", "Neha"]
C. ["Neha"]
D. Error

Negative indices count from the end of the list, so `-2` refers to "Aman".
The slice `[-2:]` means "from the second-last element to the end", returning ["Aman", "Neha"].

Q.10. If students = ["Ravi", "Anya", "Tina", "Aman", "Neha", "Kabir"], what does students[::2] return?

A. ["Ravi", "Tina", "Neha"]
B. ["Anya", "Aman", "Kabir"]
C. ["Ravi", "Anya", "Kabir"]
D. ["Ravi", "Tina", "Aman", "Kabir"]

The slice `[::2]` means "start to end, step 2".
It selects every second element, starting with index 0: "Ravi" (0), "Tina" (2), "Neha" (4).

Satish is building a backup system to store student homework submissions. Each class's submissions are stored in a list. To avoid losing data, he writes code to create backups of these lists.

At first, Satish copies the list by simply writing:

```
backup = submissions
```

But when he changes the original list, he notices that the backup also changes. Confused, he tries another method:

```
backup = submissions[:]
```

Sometimes it works. But when the list has other lists inside (like group submissions), even that doesn't behave as he expects.

Now Satish is unsure — is he really creating a new list each time?

Q.11. Satish creates a backup like this:

```
submissions = ["Ravi", "Tina"]
backup = submissions
submissions.append("Aman")
```

What could explain why both submissions and backup seem to have 3 items now?

A. Satish created a reference, not a real copy.
B. The backup was overwritten by append().
C. backup and submissions point to the same list in memory.
D. The append() affected only submissions, not backup.

- **A** describes the conceptual error — he didn't make a real copy.
- **C** explains the technical reason — both variables reference the same memory location.

Q.12.

```
submissions = ["Ravi", "Tina", "Aman"]
backup = submissions[:]
submissions[0] = "Neha"
```

How does this change affect the original and backup lists?

A. backup[0] still holds "Ravi".
B. Slicing created a new list with separate memory.
C. Changing submissions[0] also updates backup[0].
D. Both lists still refer to the same object.

Using `backup = submissions[:]` creates a shallow copy, which is a new list. Changing `submissions[0]` to "Neha" does not affect the backup list, which retains the original "Ravi" at index 0.

Q.13.

```
submissions = [["Ravi", "PDF"], ["Tina", "DOC"]]
backup = submissions
submissions[0][1] = "ZIP"
```

What might Satish notice wrt the above code ?

A. The backup also shows "ZIP" for Ravi.
B. Changing the submissions list also changes the backup.
C. The backup list changed because backup and submissions both point to the same list in memory.
D. The change only applies to submissions, not backup.

- **A** describes the observable effect.
- **B** states the general behavior.
- **C** explains the technical reason (shared reference).

Q.14. Satish tries slicing again with nested lists:

```
submissions = [["Ravi", "PDF"], ["Tina", "DOC"]]
backup = submissions[:]
submissions[1][1] = "ZIP"
```

What unexpected behavior might confuse Satish?

A. The backup also shows "ZIP" instead of "DOC".
B. Slicing didn't protect inner lists from being shared.
C. The backup list is completely unrelated to the original.
D. Only the outer list was copied, not the inner lists.

- **A** describes the visible outcome: the change in the nested list appears in both `submissions` and `backup`.
- **B** and **D** explain the cause: slicing `[:]` creates a shallow copy, so the outer list is new, but the inner lists (elements) are still references to the same nested lists in memory.

Q.15. Satish makes a backup and modifies it:

```
submissions = ["Ravi", "Tina"]
backup = submissions[:]
backup.append("Neha")
```

What is the final state of both lists?

A. submissions has 2 items.
B. backup has 3 items.
C. "Neha" appears in both lists.
D. The original list was also updated with "Neha".

Since `backup = submissions[:]` creates a shallow copy (new list), appending "Neha" to `backup` only affects the backup list.
The original `submissions` list remains unchanged with its original 2 items.

Q.16. Satish creates two sublists using the same variable:

```
inner = ["Ravi"]
submissions = [inner, inner]
submissions[0].append("PDF")
```

What will submissions look like?

A. [["Ravi", "PDF"], ["Ravi"]]
B. [["Ravi", "PDF"], ["Ravi", "PDF"]]
C. [["PDF"], ["PDF"]]
D. [["Ravi"], ["Ravi", "PDF"]]

Both elements in `submissions` are references to the same list object `inner`. Appending "PDF" to `submissions[0]` modifies the shared `inner` list, so the change is visible through both references in `submissions`.

Q.17. Satish updates the backup list thinking it's separate from the original:

```
submissions = ["Ravi"]
backup = submissions
backup[0] = "Neha"
```

What will be the value of submissions[0]?

A. "Ravi"
B. ["Ravi"]
C. "Neha"
D. It raises an error

@Exam Buddy

The assignment `backup = submissions` makes both variables reference the same list. Changing `backup[0]` to "Neha" directly modifies the shared list, so `submissions[0]` also becomes "Neha".

Q.18. Satish tries this slice:

```
submissions = ["Ravi", "Tina", "Aman"]
backup = submissions[2:1]
```

What is the value of backup?

A. ["Aman", "Tina"]
B. ["Tina"]

C. []
D. It raises an error

The slice `[2:1]` has a start index (2) greater than the stop index (1).
In Python, this returns an empty list because the slice cannot be traversed forward.

Q.19. Satish modifies the backup reference like this:

```
submissions = ["Ravi", "Tina"]
backup = submissions
backup = ["Aman"]
```

What is the final value of submissions?

A. ["Aman"]
B. ["Ravi", "Tina"]
C. []
D. ["Ravi"]

The line `backup = submissions` makes `backup` point to the same list as `submissions`.
However, `backup = ["Aman"]` reassigns the variable `backup` to a new list, leaving the original `submissions` list unchanged.

Q.20. Satish slices a list of lists and modifies the first sublist:

```
group1 = [["Ravi"], ["Tina"]]
group2 = group1[:]
group1[0].append("Submitted")
```

What is the value of group2[0]?

A. ["Ravi"]
B. ["Submitted"]
C. ["Ravi", "Submitted"]
D. It raises an error

The slice `group1[:]` creates a shallow copy, so `group2` is a new outer list. However, the inner lists are still shared references, so appending "Submitted" to `group1[0]` also affects `group2[0]`.

At IIT Ropar, students loved their lunchtime. But with hundreds of students and dozens of meal combinations, chaos ruled the canteen. To solve this, the institute launched **Smart Canteen 2.0**, a digital system built entirely using Python ,with the help of five student programmers.

Each student took on a part of the system:

Nishanur: The Pairing Expert (zip())

Nishanur had to match student names with the meals they ordered. He had two lists , one of names and one of meals. At first, he used a loop with indexing to pair them, but it was messy.

Then, he discovered:

```
zip(names, meals)
```

This zipped them together into clean pairs like ("Ravi", "Idli"), ("Anya", "Poha").

Maria : The Counter (enumerate())

Maria wanted to add serial numbers while displaying the orders. She was looping with range(len(list)), until she found:

```
enumerate(names)
```

This gave her both the index and the item in a cleaner way — like (0, 'Ravi'), (1, 'Anya').

Saujanya: **The Calculator (map())**

Saujanya had to add **18% GST** to every meal price. Instead of a loop with manual math, she used:

```
map(lambda price: price * 1.18, prices)
```

This means the same calculation (adding 18% GST) is applied to each price in the list, one by one, automatically, without writing a loop. The result is a new list with all the updated prices.

Aashik: The Filter Master (filter())

Aashik needed to show only vegetarian meals. He created a list with all meals and used:

```
filter(lambda meal: meal in veg_items, all_meals)
```

This picked only the meals that matched his condition.

map() applies a function to all items in a list.

What is lambda?

A lambda is a quick, one-line function used without def. For example:

```
lambda x: x * 2
```

which is equivalent to :

```
def double(x): return x * 2
```

Their mentor, the professor, smiled and said:

Q.21. Suppose names = ["Ravi", "Anya"] and meals = ["Idli", "Poha"]. What will the following expression do?

```
zip(names, meals)
```

A. It pairs each name with a corresponding meal
B. It changes the original names and meals lists
C. It creates a sequence of grouped elements
D. It returns a list of values from the second list only

A: It pairs each name with a corresponding meal and **C: It creates a sequence of**

grouped elements are both accurate descriptions.

- **A** describes the practical outcome of the `zip()` function.
- **C** describes the technical result — it creates an iterator of tuples where elements from each input iterable are grouped together.

Q.22. Which of the following advantages of enumerate() are clearly mentioned or shown in the comprehension?

A. Automatically gives both index and value
B. Helps avoid using range(len(...))
C. Returns reversed values
D. Removes duplicate items from the list

The `enumerate()` function provides a counter (index) along with each element during iteration.
This eliminates the need for `range(len(...))` and manual indexing, making code cleaner and more Pythonic.

Q.23. Which of the following best describes a scenario where using map() would be preferred over a for loop?

A. When you want to pair student names with their meals
B. When you want to display items along with their serial numbers
C. When you want to apply a calculation to every item in a list without modifying the original
D. When you want to remove all non-vegetarian meals from a list

`map()` is ideal for applying the same function to all elements of an iterable.
It returns a new iterator with the transformed values, leaving the original list unchanged, which is more efficient and functional than a manual `for` loop for this purpose.

Q.24. What will be the output of the following code?

```
names = ['Ravi', 'Anya', 'Sita']
meals = ['Idli', 'Poha', 'Dosa']
pairs = zip(names, meals)
names[0] = 'Amit'
print(list(pairs))
```

A. [('Ravi', 'Idli'), ('Anya', 'Poha'), ('Sita', 'Dosa')]
B. [('Amit', 'Idli'), ('Anya', 'Poha'), ('Sita', 'Dosa')]
C. [('Amit', 'Poha'), ('Anya', 'Dosa')]
D. [('Amit', 'Poha'), ('Sita', 'Dosa')]

The `zip()` function creates an iterator that generates tuples on-the-fly when `list(pairs)` is called.
However, the `zip` object is created *before* `names[0]` is changed to 'Amit', so it uses the original 'Ravi' value.

Q.25. What is the output of this code?

```
data = ['Idli', 'Poha', 'Dosa']
for i, item in enumerate(data):
    data[i] = item.upper()
print(data)
```

A. ['Idli', 'Poha', 'Dosa']
B. ['IDLI', 'POHA', 'DOSA']
C. ['IDLI', 'Poha', 'Dosa']
D. ['Idli', 'POHA', 'DOSA']

The `enumerate()` loop provides both index `i` and value `item` for each element.
Inside the loop, `data[i] = item.upper()` replaces each element with its uppercase version, modifying the original list.

Q.26. Predict the output:

```
prices = [100, 200]
updated_prices = list(map(lambda p: p * 1.18, prices))
prices[1] = 300
print(updated_prices)
```

A. [118.0, 236.0]
B. [118.0, 354.0]
C. [100, 300]
D. [118, 300]

The `map()` function is executed immediately when `list(map(...))` is called, creating `updated_prices` from the original `prices` values [100, 200].
Changing `prices[1]` to 300 afterward does not affect the already-computed `updated_prices` list.

@Exam

Q.27. What will the following code return?

```
meals = ['Idli', 'Poha', 'Chicken']
veg_items = ['Idli', 'Poha', 'Paneer']
result = filter(lambda x: x not in veg_items, meals)
print(len(list(result)))
```

A. 0
B. 1
C. 2
D. 3

The `filter()` function keeps only items from `meals` that are NOT in `veg_items`.
From `meals = ['Idli', 'Poha', 'Chicken']`, only 'Chicken' is not in `veg_items`, so the filtered list has 1 element.

Q.28. What will be the final content of new_prices?

```
meal_prices = [100, 150, 200]
gst_func = lambda p: p * 1.18
new_prices = list(map(gst_func, meal_prices[:2]))
meal_prices[1] = 300
print(new_prices)
```

A. [118.0, 177.0]
B. [118.0, 354.0]
C. [118.0, 177.0, 236.0]
D. [118.0, 300.0]

The `map()` applies the GST function to a slice `meal_prices[:2]` which is `[100, 150]`.
This creates `new_prices = [118.0, 177.0]`. Changing `meal_prices[1]` later to 300 does not affect the already computed `new_prices`.

Q.29. What will be printed?

```
data = [10, 20, 30]
for i, val in enumerate(data):
    val = val + 5
print(data)
```

A. [15, 25, 35]
B. [10, 20, 30]
C. [10, 25, 30]
D. Error

Inside the loop, `val` is a local variable that gets modified (`val = val + 5`), but this does not update the original list `data`.
The `enumerate()` provides values, not direct references to list elements, so the list remains unchanged.

Q.30. Choose the correct output:

```
items = ['a', 'b', 'c']
result = list(map(lambda x: x + '1', items))
print(''.join(result))
```

A. abc
B. ['a1', 'b1', 'c1']
C. a1b1c1
D. Error

The `map()` function applies `lambda x: x + '1'` to each element, creating `['a1', 'b1', 'c1']`.
The `''.join(result)` concatenates all elements into a single string: `'a1b1c1'`.

@Exam Buddy

Week 4 Assignment

During the weekend lab, students in the Joy of Computing course created a mini game called Guess the Character. The idea is simple: take a famous movie or cartoon character's name, shuffle the letters, and let the user guess who it is.

But instead of doing this manually every time, the instructor asked the students to automate it using Python. Here's the code skeleton they were given:

```python
def jumble_word(word):
    word_letters = list(word)
    random.shuffle(word_letters)
    return ''.join(word_letters)

def guess_the_character():
    characters = ["Ironman", "Elsa", "Yoda", "Moana", "Harry", "Simba"]
    original = random.choice(characters)
    jumbled = jumble_word(original)

    print("Guess the character from this jumble: ", jumbled)

    guess = input("Your guess: ")

    if guess.lower() == original.lower():
        print("Correct!")
    else:
        print("Oops! The correct answer was:", original)


guess_the_character()
```

Q.1. What is the output type of the function jumble_word("Moana")?

A. List
B. Tuple
C. Dictionary
D. String

The function `jumble_word` converts the input word to a list, shuffles the letters, and then uses `''.join()` to combine them back into a string.
The final `return` statement outputs a string.

Q.2. Why do we use list(word) before shuffling in jumble_word?

A. Strings are immutable, so we convert to list
B. Strings can be shuffled directly
C. It converts all characters to uppercase
D. It reverses the word

The `random.shuffle()` function works by modifying a list in-place.
Since strings in Python cannot be changed (immutable), we must convert the string to a list of characters first.

Q.3. Consider the line: if guess.lower() == original.lower():

What will happen if we remove .lower() from both sides?

A. It will make the comparison more accurate
B. The game will stop working entirely
C. It may incorrectly mark right answers as wrong if case differs
D. The original variable will change

The `.lower()` method ensures case-insensitive comparison.
Without it, "elsa" would not match "Elsa", causing correct guesses with wrong case to be marked as wrong.

@Exam Buddy

Q.4. Which function is used to randomize the order of items in a list in the given code?

A. random.choice()
B. random.shuffle()
C. random.sort()
D. random.sample()

The code explicitly uses `random.shuffle(word_letters)` to randomize the order of letters in the list.
This function modifies the list in-place, rearranging its elements randomly.

Q.5. What issue can occur with this line: jumbled = jumble_word(original)?

A. It may return the original word in the same order
B. It will always return the original word
C. It causes a syntax error
D. It deletes a letter from the word

`random.shuffle()` randomly rearranges the list, but there is a chance (though small) that the shuffled order could be the same as the original.
This means the jumbled word could occasionally be identical to the original word.

Q.6. Which modification ensures that the jumbled word is not the same as the original?

A. Add .lower() after shuffle()
B. Convert it to uppercase
C. Use reverse() instead of shuffle()
D. Add a loop to reshuffle until different

This would repeatedly call `jumble_word()` until the returned jumbled word is not equal to the original.
It guarantees the jumbled version is always different from the original word.

@Exam Buddy

Q.7. Suppose the word is “Yoda”. Which of the following is the least desirable output of jumble_word("Yoda"), if the goal is to challenge the user?

A. "odYa"
B. "Yoda"
C. "dayo"
D. "odaY"

If the jumbled output is identical to the original word "Yoda", it provides no challenge to the user.
The goal is to scramble the letters, so an unchanged output defeats the purpose of the game.

Q.8. What is the purpose of random.choice(characters)?

A. To select a character randomly from the list
B. To jumble all characters
C. To sort the characters alphabetically

D. To pick all characters one by one

The `random.choice()` function randomly selects one element from the given list `characters`.
This ensures a different character is chosen each time the game is played.

Q.9. What will happen if the characters list is empty?

A. The program will crash with IndexError
B. The program will continue running without error
C. The program will print a blank string
D. The program will crash with ValueError

Calling `random.choice()` on an empty list raises a `ValueError` because there are no items to choose from.
The error message would be: "Cannot choose from an empty sequence".

@Exam Buddy

Q.10. Which of the following is the best way to score the user if you want to turn this into a 5-round game?

A. Add 5 to score after every round
B. Create a list of 5 questions and loop with a score variable
C. Print the score inside the jumble_word function
D. Use break instead of score

This approach would run the game for 5 rounds, track correct guesses with a score counter, and provide a final score at the end.
It structures the game into multiple rounds with proper scoring.

In Python, a matrix is not a special built-in type. It is usually represented as a list of lists, where each inner list represents a row of the matrix.

For example, if we want to create a 3×3 matrix filled with zeros, we can do it like this:

```
matrix = []

for row in range(3):
    new_row = []
    for col in range(3):
        new_row.append(0)
    matrix.append(new_row)

for row in matrix:
    print(row)
```

This code will output:
[0, 0, 0]
[0, 0, 0]
[0, 0, 0]

Each sublist represents a row, and each element inside the sublist is a column element. Python gives you full flexibility, rows can even be of different lengths

Q.11. What will be the output of the following code?

```
matrix = []
row = [0] * 3
for i in range(3):
    matrix.append(row)

matrix[0][0] = 99
print(matrix)
```

A. [[99, 0, 0], [99, 0, 0], [99, 0, 0]]
B. [[99, 0, 0], [0, 0, 0], [0, 0, 0]]
C. [[99, 0, 0], [99, 0, 0], [0, 0, 0]]
D. [[0, 0, 0], [0, 0, 0], [0, 0, 0]]

The code appends the same `row` list object to `matrix` three times.

Changing `matrix[0][0]` modifies this shared list, so the change appears in all three rows.

Q.12. What does line matrix.append([0] * 3)do inside a loop?

A. Adds the same list multiple times
B. Creates a new list each time and appends it
C. Appends 0 three times
D. Adds a tuple to the matrix

The expression `[0] * 3` creates a new list `[0, 0, 0]` each time the loop runs. Appending it to `matrix` ensures each row is a separate list, avoiding the shared reference issue.

Q.13. What is the result of this code?

```
matrix = [[i + j for j in range(3)] for i in range(3)]
print(matrix[2][1])
```

A. 1
B. 2
C. 3
D. 4

Then `matrix[2][1]` would be:

- For i=2, j=1: 2 + 1 = **3**

Q.14. Which of the following correctly creates a 2×2 matrix of zeros?

A. [[0, 0], [0, 0]]
B. [0, 0, 0, 0]

C. [[0]*2]*2
D. Both a and c

- **a:** `[[0, 0], [0, 0]]` is a literal 2x2 matrix.
- **c:** `[[0]*2]*2` creates a 2x2 structure, but note: the inner lists are the same object (shared reference), so it has the same issue as earlier examples. Structurally, it's still a 2x2 matrix of zeros.

Q.15. Which of these statements access the last element of the first row , in the matrix [[0, 0, 0] [0, 0, 0] [0, 0, 0]]

A. matrix[0][0]
B. matrix[1][2]
C. matrix[0][2]
D. matrix[2][0]

- `matrix[0]` accesses the first row `[0, 0, 0]`.
- `[2]` accesses the third element (index 2), which is the last element of that row.

Q.16. What does the following code do?

```
matrix = []
for i in range(2):
    row = []
    for j in range(3):
        row.append(i + j)
    matrix.append(row)
```

A. Creates a 2×3 matrix with each row filled with i
B. Adds only one row
C. Creates a 2×3 matrix with values increasing across rows
D. Creates a 3×2 matrix

The outer loop runs 2 times (i=0,1), creating 2 rows.

The inner loop runs 3 times (j=0,1,2), filling each row with values `i+j`, so the matrix will be:
[[0, 1, 2], [1, 2, 3]].

Q.17. Which Python structure is best to represent a 3×3 grid of numbers?

A. List of strings
B. List of tuples
C. List of lists
D. Dictionary

A list of lists allows each inner list to represent a row, providing easy access to elements using double indexing like `grid[row][col]`.
This structure is mutable and flexible, making it ideal for representing matrices or grids.

Q.18. What does this code do?

```
matrix = [[1]*3 for i in range(3)]
```

A. Creates a 1×3 matrix
B. Creates a 3×3 matrix with all values as 1
C. Gives an error
D. Creates a 3×1 matrix

The list comprehension `[[1]*3 for i in range(3)]` creates 3 rows.
Each row is a new list `[1, 1, 1]`, resulting in a 3x3 matrix filled with 1s.

Q.19. Which of the following will raise an IndexError for a 2×2 matrix?

A. matrix[1][1]
B. matrix[2][0]
C. matrix[0][1]
D. matrix[1][0]

A 2×2 matrix has valid row indices 0 and 1.
Accessing `matrix[2]` tries to access a third row, which doesn't exist, causing an `IndexError`.

Q.20. What will be the output of the following code?

```
matrix = [[1, 2, 3],
          [4, 5, 6]]

result = []

for col in range(3):
    temp = []
    for row in range(2):
        temp.append(matrix[row][col])
    result.append(temp)

print(result)
```

A. [[1, 4], [2, 5], [3, 6]]
B. [[1, 2, 3], [4, 5, 6]]
C. [[1, 4, 2, 5, 3, 6]]
D. [[1, 2], [4, 5], [3, 6]]

This code transposes the matrix (swaps rows and columns).
It iterates through columns first, collecting elements from each row to form new rows in `result`, converting the 2×3 matrix into a 3×2 matrix.

Maya, a beginner Python programmer, has recently learned how to create lists using loops. Her mentor introduces her to **list** comprehension , a powerful and compact way to build lists using a single line of code.

Maya is especially interested in 2D lists (matrices) and wants to:

Create a matrix of zeroes
Generate addition tables
Fill custom values using formulae
Perform transposition
Apply filtering (like collecting even numbers)

Her mentor shows her the following code snippet:

```
zeros = [[0 for col in range(3)] for row in range(3)]

add_table = [[r + c for c in range(3)] for r in range(3)]

rows, cols = 2, 3
matrix = [[r * 10 + c for c in range(cols)] for r in range(rows)]

transpose = [[matrix[r][c] for r in range(rows)] for c in range(cols)]

evens = [x for x in range(10) if x % 2 == 0]
```

Q.21. What is the value of zeros[1][2]?

A. 1
B. 0
C. 2
D. Error

@Exam Buddy

The list comprehension `[[0 for col in range(3)] for row in range(3)]` creates a 3x3 matrix filled with zeros.
Thus, `zeros[1][2]` accesses the element at row 1, column 2, which is 0.

Q.22. What is the length of the list evens?

A. 5
B. 10
C. 0
D. 9

The list comprehension `[x for x in range(10) if x % 2 == 0]` collects even numbers from 0 to 9.
The even numbers are: 0, 2, 4, 6, 8 → total 5 elements.

Q.23. What is the type of add_table?

A. Integer
B. List
C. String
D. Dictionary

`add_table` is created using nested list comprehension: `[[r + c for c in range(3)] for r in range(3)]`.
This constructs a list of lists, making `add_table` a list type.

@Exam Buddy

Q.24. What does matrix[1] evaluate to?

A. [10, 11, 12]
B. [0, 1, 2]
C. [1, 2, 3]
D. [11, 12, 13]

The list comprehension `[[r * 10 + c for c in range(cols)] for r in range(rows)]` with `rows=2, cols=3` creates:

- For r=0: [0, 1, 2]
- For r=1: [10, 11, 12]

Q.25. What is the shape (rows × cols) of the transpose list?

A. 2 × 3
B. 3 × 2
C. 2 × 2

D. 3 × 3

The original `matrix` has shape 2×3 (2 rows, 3 columns).
The transpose swaps rows and columns, so `transpose` has shape 3×2 (3 rows, 2 columns).

Q.26. What is the value of add_table[2][2]?

A. 2
B. 4
C. 5
D. 6

`add_table = [[r + c for c in range(3)] for r in range(3)]`
For `r=2` and `c=2`: `2 + 2 = 4`.
So `add_table[2][2]` is **4**.

@Exam Buddy

Q.27. What is the value of transpose[0][1]?

A. 10
B. 0
C. 1
D. 11

`transpose = [[matrix[r][c] for r in range(rows)] for c in range(cols)]`

- `transpose[0][1]` means column 0, row 1 of the original matrix.
- Original `matrix[1][0]` is `1 * 10 + 0 = 10`.

Q.28. What is the last element of the list evens?

A. 10
B. 8
C. 9
D. 6

The list `evens` contains even numbers from `range(10)` (0 to 9).
The even numbers are: 0, 2, 4, 6, 8.
The last element is **8**.

Q.29. Which formula was used to generate the values in the matrix list?

A. r + c
B. r * c
C. r * 10 + c
D. r + c * 10

The code explicitly shows:
`matrix = [[r * 10 + c for c in range(cols)] for r in range(rows)]`
This formula generates values like 10, 11, 12 for row 1.

@Exam Buddy

Q.30. What is the sum of all values in transpose[1]?

A. 11
B. 12
C. 13
D. 114

`transpose` swaps rows and columns:

- Column 0 becomes row 0: [0, 10]
- Column 1 becomes row 1: [1, 11]
- Column 2 becomes row 2: [2, 12]

`transpose[1]` is [1, 11].
Sum = 1 + 11 = **12**.

© Exam Buddy → NPTEL- JULY 2025 Course Notes Gmail – nptelsolver@gmail.com



@Exam Buddy

Week 6 Assignment

In school mathematics, we're used to writing equations like:

f(x) = x + 2

This means: "Give me a number x, and I'll give you back x + 2."

Python functions work **exactly like this**, but they need a little more structure so the computer understands.
Let's explore how a machine (Python) thinks about functions

```
def f(x):
    return x + 2
```

She says:
“This is how you tell the machine: when I give you a number x, return x + 2.”

- def means “define a function”
- f(x) means the function's name is f, and it needs one input
- return tells Python what to give back.

So, calling: f(5) will return 7.

This is just like math! But now, you can teach a machine to remember this rule.

More Rules: Multiple Inputs

What if you want to define a function like:

g(x, y) = x * y + 3

In Python:

```
def g(x, y):
    return x * y + 3
```

@Exam Buddy

Now, you can ask:

g(2, 4)

The function can have **multiple inputs**, and it still returns one output.

Making Reusable Tools

Instead of writing the same calculation again and again, students build small functions like:

```
def square(n):
  return n * n
```

0r

```
def is_even(num):
  return num % 2 == 0
```

Bonus Tools: Methods vs Attributes

The PyCalc is upgraded with a NumPy brain to handle data tables (arrays).

The teacher loads:

```
import numpy as np
data = np.array([[1, 2], [3, 4]])
```

This looks like a matrix or table from math class.

To find out its shape:

```
print(data.shape)      # prints (2, 2)
```

Here, .shape is an attribute — a fact about the object.

But to reshape it into a new form:

```
reshaped = data.reshape((4, 1))
```

.reshape() is a method — a function attached to an object, which performs an action.
How to tell the difference?

- No () → attribute → describes something
- With () → method → does something

Q.1.

```
def parity(n):
  return n % 2
```

Which of the following options correctly represents the possible outputs of the function defined above ?

A. 0,1
B. even, odd
C. 5,6
D. 2,1

The function `parity(n)` returns `n % 2`, which is the remainder when `n` is divided by 2. For any integer `n`, the result can only be **0** (if even) or **1** (if odd).

Q.2. Consider:

```
def f(x):
  return x * 3 + 1
```

What is the result of:

```
print(f(1), f(2), f(3))
```

A. 4 7 10
B. 4 6 8
C. 3 6 9
D. 1 2 3

`print(f(1), f(2), f(3))` outputs **4 7 10**.

Q.3. Observe the following:

```
def mystery_function(x):
  return 5
```

What **type of mathematical function** does mystery_function represent?

A. **One-to-one**, since different inputs give different outputs
B. **Onto**, because all values in the range are covered
C. **Many-to-one**, since multiple inputs map to the same output
D. **Bijective**, because the function is both one-to-one and onto

The function always returns 5 regardless of the input value `x`.
This means many different inputs all map to the same single output, making it a many-to-one function.

Q.4. A student defines the following function to simulate a formula from math class:

```
def weighted_sum(x, y=2, z=3):
    return x + y * z
```

They then execute:

```
result = weighted_sum(1, z=4)
print(result)
```

What will be the output?

A. 9
B. 21
C. 11
D. Error: positional argument follows keyword argument

The function call `weighted_sum(1, 2, z=4)` provides values for all parameters: `x=1`, `y=2`, `z=4`.
The calculation is `x + y * z = 1 + (2 * 4) = 1 + 8`.
The result is **9**.

@Exam Buddy

Q.5. In the PyCalc system, two functions are defined:

```
def double(x):
    return x * 2

def add_three(x):
    return x + 3
```

A student composes them like this:

```
def composed(x):
    return add_three(double(x))
```

What is the output of composed(4)?

A. 11
B. 14
C. 7
D. 24

The composition applies `double(4)` first, which returns 8.
Then `add_three(8)` is called, returning 11.
So `composed(4)` outputs **11**.

Q.6. The PyCalc team loaded a NumPy array and wrote the following code:

```
import numpy as np
data = np.array([[10, 20], [30, 40]])
```

A student is asked to print the shape of the array and reshape it into 4 rows and 1 column. They write:

```
print(data.shape())
reshaped = data.reshape(4, 1)
print(reshaped.shape)
```

What will happen when this code is executed?

A. It prints (2, 2) and (4, 1)
B. It raises an error on line 1 because reshape cannot be used this way
C. It raises an error because .shape is not a function and cannot be called with ()
D. It raises an error because reshaping changes the original array

The `.shape` attribute of a NumPy array is not a method, so using `data.shape()` with parentheses causes an `AttributeError`.
The correct syntax is `data.shape` without parentheses.

Q.7.

```
def manual_sum(lst):
    total = 0
    for i in lst:
        total = total + i
    return total

def process_list(L):
    squares = []
    for item in L:
        squares.append(item ** 2)
    return squares, manual_sum(squares)

result = process_list([1, 2, 3])
print(result[0][1], result[1])
```

What will be printed?

A. 2 6
B. 1 6
C. 4 14
D. 9 6

The function `process_list([1, 2, 3])` returns `([1, 4, 9], 14)` where 14 is the sum of squares.
`result[0][1]` accesses the second element `4` from the squares list `[1, 4, 9]`.

`result[1]` is the sum `14`, so the output is **4 14**.

Q.8.

```
def identity():
  return "Function called"
identity_value = "Just a value"
```

Later, they write the following code:

```
print(identity)
print(identity())
print(identity_value())
```

What will happen when this code is run?

Function called
Just a value

<function identity at ...>
Function called
Error: 'str' object is not callable

<function identity at ...>
<function identity at ...>
Just a value

Function called
Error
Error

@Exam Buddy

`print(identity)` prints the function object itself. `print(identity())` calls the function and prints its return value "Function called". `print(identity_value())` tries to call a string as a function, causing an error.

Q.9. A student is trying to understand the difference between variables and functions in Python. They write the following:

```
def message():
    return "This is a function"

message = "This is a variable"

print(message())
```

What will happen when this code runs?

A. This is a function
B. This is a variable
C. Error: 'str' object is not callable
D. None

The code first defines `message` as a function, then reassigns `message` to a string. When `message()` is called, Python tries to call the string as a function, which causes the error.

Q.10. A student defines the following function to collect error logs during program execution:

```
def log_error(msg, log=[]):
    log.append(msg)
    return log
```

They run the following code:

```
print(log_error("E1"))
print(log_error("E2"))
print(log_error("E3", []))
```

What will be the output?

['E1']
['E2']
['E3']

['E1']
['E1', 'E2']
['E3']

['E1']
['E1']
['E3']

['E1']
['E2']
['E3', 'E1']

First call: `log_error("E1")` uses the default mutable `log=[]`, appends "E1", returns `['E1']`Second call: uses the same default list (now `['E1']`), appends "E2", returns `['E1', 'E2']`.Third call: provides new `log=[]`, appends "E3", returns `['E3']`

In computer science, searching algorithms are fundamental techniques used to locate specific data within a collection, such as an array or list.

Linear Search is the simplest searching algorithm. It works by checking each element one by one, starting from the beginning of the list, until the desired
element is found or the end of the list is reached. It does not require the list to be sorted.

Here is a Python implementation of linear search:

```python
def linear_search(arr, target):
    for index in range(len(arr)):
        if arr[index] == target:
            return index
    return -1
```

In contrast, **Binary Search** is a more efficient algorithm, but it only works on **sorted arrays**. It repeatedly divides the search interval in half. If the target value is less than the element in the middle, the search continues in the left half. Otherwise, it continues in the right half. This continues until the target is found or the interval is empty.

Here is a Python implementation of binary search:

```python
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1
```

Q.11. What is the time complexity of the worst-case scenario for linear search?

A. O(1)
B. O(log n)
C. O(n)
D. O(n log n)

@Exam Buddy

In the worst case, linear search checks every element in the array once.
The time grows linearly with the input size n, giving O(n) complexity.

Q.12. Which condition must be met for binary search to function correctly?

A. The array must have unique elements
B. The array must be unsorted
C. The array must be sorted
D. The array size must be even

Binary search works by repeatedly dividing the search interval in half.
This divide-and-conquer approach only works if the array is sorted in ascending or descending order.

Q.13. What will be the output of binary_search([1, 3, 5, 7, 9], 7)?

A. 2
B. 3
C. 4
D. 1

The sorted array is `[1, 3, 5, 7, 9]` and target is `7`.

- First mid: (0+4)//2 = 2 → value 5 (too low) → low = 3
- Second mid: (3+4)//2 = 3 → value 7 (match)
  Returns index **3**.

Q.14. In the binary search algorithm, what happens if arr[mid] > target?

A. Search left half of the array
B. Search right half of the array
C. Return mid
D. Return -1

@Exam Buddy

If `arr[mid] > target`, the target must be in the lower half of the current search range. The algorithm sets `high = mid - 1` to search the left subarray.

Q.15. In the following code, what value is returned?

```python
def linear_search(arr, target):
    for index in range(len(arr)):
        if arr[index] == target:
            return index
    return -1
arr = [10, 20, 30, 40]
print(linear_search(arr, 25))
```

A. 2
B. 3
C. -1
D. Error

The target `25` is not present in the array `[10, 20, 30, 40]`.
The linear search checks all elements, finds no match, and returns `-1` as specified.

@Exam Buddy

Q.16. For a sorted list of 1,000,000 elements, which search algorithm is generally faster for finding a value in terms of average-case performance?

A. Linear Search
B. Binary Search
C. Both are equally fast
D. Performance depends mainly on the programming language

Binary search has O(log n) time complexity, requiring only about 20 comparisons for 1,000,000 elements.
Linear search has O(n) complexity, requiring up to 1,000,000 comparisons on average.

Q.17. Using the binary_search implementation above, what is the maximum number of times the comparison arr[mid] == target can be executed when searching a sorted list of 256 elements?

A. 256

B. 128
C. 8
D. 9

For a list of 256 elements, binary search halves the search space each time.
The maximum number of comparisons is $\lfloor \log_2(256) \rfloor + 1 = 8 + 1 = \mathbf{9}$ (including the final successful check).

Q.18. Which statement is FALSE?

A. Linear search does not need sorted data
B. Binary search needs sorted data
C. Binary search always performs better than linear search
D. Linear search checks every element until it finds the match

This is FALSE because for very small arrays or when the target is at the beginning, linear search can be faster.Binary search has better asymptotic complexity (O(log n) vs O(n)), but it's not always faster in practice for every scenario.

@Exam Buddy

```python
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1
arr = [3, 6, 9, 12, 15]
print(binary_search(arr, 4))
```

Q.19. What will be the output of the following code?

A. 0
B. -1
C. 1
D. None

The target `4` is not in the sorted array `[3, 6, 9, 12, 15]`.
The binary search will complete without finding the target and return `-1`.

Q.20. If the element you are searching for is known to be at the end of a 100-element list and you use linear search, how many comparisons will it take?

A. 100
B. 50
C. 25
D. 10

In the worst-case scenario for linear search, the target is at the end of the list.
The algorithm must check all 100 elements before finding the target at the last position.

The **Monty Hall Problem** is a famous probability puzzle based on a game show scenario. It presents a counterintuitive result that surprises many people when they first encounter it.

The Problem Setup:

You are a contestant on a game show. There are **three doors:**

- Behind **one door** is a **car** (the prize).
- Behind the other **two doors** are **goats**.

The game proceeds as follows:
1. You choose one of the three doors.
2. The host (Monty Hall), who **knows what's behind the doors**, opens one of the **other two doors** to reveal a **goat**.
3. You are then given a choice: **stick** with your original choice, or **switch** to the remaining unopened door.

Should You Switch or Stay?

Surprisingly, **switching gives you a higher chance of winning** the car.

- If you **stick**: your chance of winning = **1/3**
- If you **switch**: your chance of winning = **2/3**

This result can be verified through **simulation** using lists in Python.

```python
import random

def simulate_monty_hall(switch: bool) -> bool:
    doors = ['goat', 'goat', 'car']
    random.shuffle(doors)  # Randomize door positions

    # Contestant makes a random choice
    choice = random.randint(0, 2)

    # Host opens a door with a goat (not the chosen one)
    available_doors = [i for i in range(3) if i != choice and doors[i] == 'goat']
    monty_opens = random.choice(available_doors)

    # If switch is True, change the choice to the remaining unopened door
    if switch:
        remaining_doors = [i for i in range(3) if i != choice and i != monty_opens]
        choice = remaining_doors[0]

    return doors[choice] == 'car'
```

You can simulate this many times to check the **probability of winning** by switching vs. staying:

```python
trials = 10000
switch_wins = sum(simulate_monty_hall(True) for _ in range(trials))
stay_wins = sum(simulate_monty_hall(False) for _ in range(trials))

print(f"Win percentage when switching: {switch_wins / trials * 100:.2f}%")
print(f"Win percentage when staying: {stay_wins / trials * 100:.2f}%")
```

Q.21. In the Monty Hall simulation, why does switching give a higher probability of winning the car, even though there are only two doors left at the end?

A. The host reveals the car
B. The initial choice had a 1/3 chance of being correct, so switching captures the remaining ⅔
C. The remaining door is chosen randomly
D. All options become equally likely after Monty's reveal

Initially, your choice has a 1/3 probability of being the car.
When Monty reveals a goat from the other two doors, switching doors gives you the combined 2/3 probability from the two unchosen doors.

Q.22. In the following Python code, what does the line marked with # ensure ?

```
available_doors = [i for i in range(3) if i != choice and doors[i] == 'goat']  #
monty_opens = random.choice(available_doors)
```

A. It guarantees the host never reveals the car
B. It helps the contestant win more often
C. It randomly opens a car door
D. It changes the position of the car

The list comprehension filters to only include doors that are **not the contestant's choice** and contain **goats**.
This ensures Monty always reveals a goat door, never the car, following the game's rules.

Q.23. What would happen if the list available_doors included doors that might have the car behind them?

A. The switch strategy would always win
B. The simulation would stop with an error
C. The probability advantage of switching would no longer hold
D. It would make the list empty

If Monty could reveal the car, the game's fundamental rules would be broken.
The 2/3 probability advantage of switching depends entirely on Monty always revealing a goat.

Q.24. What is the main purpose of random.shuffle(doors) in the simulation?

A. To select a goat door
B. To ensure fairness by randomly placing the car
C. To ensure Monty opens the correct door
D. To make the output deterministic

`random.shuffle(doors)` randomizes the position of the car among the three doors at the start of each game.
This ensures the simulation is unbiased and reflects the true probabilities of the Monty Hall problem.

Q.25. Which of the following modifications would incorrectly simulate the Monty Hall problem?

A. Removing the use of random.shuffle() when placing the car behind one of the doors.
B. Allowing Monty to randomly open any unchosen door, even if it hides the car.
C. Always forcing the player to switch their initial choice after Monty opens a door.
D. Running the simulation 10,000 times to observe overall win rates.

This violates the core rule of the Monty Hall problem: the host always reveals a goat.
If Monty could reveal the car, the probability structure (1/3 vs 2/3) would be destroyed.

Q.26. What does this snippet do in the simulation?

```
remaining_doors = [i for i in range(3) if i != choice and i != monty_opens]
choice = remaining_doors[0]
```

A. Selects the door with the car
B. Randomly opens a door
C. Switches the player's choice to the remaining unopened door
D. Forces the player to lose

The code creates a list of doors that are not the initial choice and not opened by Monty.
Then it updates the choice to the only door in that list, implementing the switching strategy.

Q.27. Why is running the simulation with thousands of trials (e.g., 10,000) important?

A. To stress test the program
B. To ensure the code doesn't crash

C. To approximate the actual win probabilities using empirical evidence
D. Because smaller numbers always produce 100% win rates

Running many trials allows the Law of Large Numbers to take effect.
The observed win rates will converge to the theoretical probabilities (≈1/3 for staying, ≈2/3 for switching).

Q.28. You pick one of the three doors at random. What is the probability that your initial choice is **wrong**?

A. ⅓
B. ½
C. ⅔
D. Cannot be determined

There are 3 doors, and only 1 has the car.
The probability your initial choice is wrong (i.e., you picked a goat) is 2 out of 3.

Q.29. You pick Door 1. Monty, knowing the prize location, opens Door 3, which has a goat. What is the probability the car is behind Door 2 if you initially picked a goat?

A. 1/3
B. 1/2
C. 1
D. 2/3

If you initially picked a goat (probability 2/3), Monty is forced to reveal the only other goat.
The remaining unopened door must contain the car, so the probability is 1 (100%).

Q.30. What kind of probability concept does the Monty Hall problem mainly help us understand?

A. How to calculate the total number of possible events
B. How to identify equally likely outcomes
C. How probability changes when new information is revealed
D. How to simulate random experiments using coins and dice

The Monty Hall problem demonstrates conditional probability - how probabilities update when we gain new information (Monty revealing a goat).
It shows that initial probabilities don't always remain fixed when the situation evolves.

@Exam Buddy

Week 6 Assignment

Prof. Andrew is a computer science teacher at a high school. With the annual Parent-Teacher Meeting approaching, he wants to organize student data neatly using Python. He plans to keep track of students' marks, personal details, and house assignments.

Being a Python pro, Prof. Andrew considers two ways to represent this data effectively.

First, a Dictionary of Lists Prof. Andrew thinks to himself: *"I want to quickly check a student's marks just by typing their name."* So, he writes the following code:

```
marks = {
    "Aanya": [85, 90, 88],
    "Ravi": [78, 82, 80],
    "Zoya": [92, 95, 89]
}
```

Each key is a student's name, and the value is a list of their marks in three subjects: Math, Science, and English.

If he wants to know Ravi's Science mark (the second subject), he simply writes:

```
print(marks["Ravi"][1])
```

The output is 82.

This structure works well when he needs to look up a **specific student** and view all their marks.

Next, a List of Dictionaries

Later, the school admin asks Prof. Andrew for a list of students along with their roll numbers and house names.

He now decides to create a different structure — one where **each student is a complete record.**

```
students = [
    {"name": "Aanya", "roll": 1, "house": "Red"},
    {"name": "Ravi", "roll": 2, "house": "Blue"},
    {"name": "Zoya", "roll": 3, "house": "Green"}
]
```

Now he can loop through each student's information:

```
for student in students:
    print(student["name"], "belongs to", student["house"], "house.")
```

@Exam Buddy

This setup is ideal when working with multiple entries and needing to **process each student's data one by one** — similar to going through rows in a spreadsheet.

Q.1. What data structure did Prof. Andrew use to store student marks where the key was the student's name and the value was a list of marks?

A. List of Lists
B. List of Dictionaries
C. Dictionary of Lists
D. Dictionary of Dictionaries

The description states: "key was the student's name and the value was a list of marks." This structure uses student names as dictionary keys and their marks stored in lists as values.

Q.2. What will be the output of the following code?

```
marks = {
    "Aanya": [85, 90, 88],
    "Ravi": [78, 82, 80],
    "Zoya": [92, 95, 89]
}
print(marks["Zoya"][0])
```

A. 85
B. 92
C. 89
D. Error

`marks["Zoya"]` accesses the list `[92, 95, 89]`.
`[0]` then gets the first element of that list, which is **92**.

Q.3. Fill in the blank to print Ravi's English marks from the marks dictionary:

```
print(marks["Ravi"][___])
```

A. 2
B. 0
C. 1
D. 3

If the subjects are stored as `[Math, Science, English]`, then English is the third subject.
In a zero-indexed list, the third subject is at **index 2**.

Q.4. Which of the following best describes when to use a list of dictionaries?

A. When you want to access one student's data quickly
B. When the values are numbers only
C. When you want to loop through each student's complete record
D. When you are storing a single subject's marks

A list of dictionaries is ideal when you need to process multiple records with the same structure.
Each dictionary can represent a student with their attributes, making it easy to iterate through all students.

Q.5. What will be the output of the following code?

```
students = [
    {"name": "Aanya", "roll": 1, "house": "Red"},
    {"name": "Ravi", "roll": 2, "house": "Blue"},
    {"name": "Zoya", "roll": 3, "house": "Green"}
]

print(students[1]["house"])
```

A. Red
B. Green
C. Blue
D. Ravi

`students[1]` accesses the second dictionary in the list: `{"name": "Ravi", "roll": 2, "house": "Blue"}`.
`["house"]` then retrieves the value `"Blue"` associated with the key `"house"`.

Q.6. What type of data structure is students in the code used by Prof. Andrew?

A. Dictionary of Lists
B. Tuple of Dictionaries
C. List of Dictionaries
D. Dictionary of Tuples

The variable `students` is defined with square brackets `[ ]`, making it a list.
Each element inside is a dictionary `{ }`, so it's a list containing dictionary elements.

Q.7. What will happen if Prof. Andrew writes print(students["Ravi"]["house"])?

A. It will print "Blue"

B. It will cause an IndexError
C. It will cause a TypeError
D. It will cause a KeyError

The variable `students` is a list, not a dictionary, so you cannot use a string key like `"Ravi"` to index it.
Using `students["Ravi"]` will raise a `TypeError` because list indices must be integers, not strings.

Q.8. Prof. Andrew updates the students list by adding a new student record like this:

```
students.append({"name": "Meera", "roll": 4, "house": "Red"})
students[0]["house"] = "Yellow"
```

Now, what will be the output of the following code?

```
count = 0
for student in students:
    if student["house"] == "Red":
        count += 1
print(count)
```

A. 2
B. 1
C. 3
D. 0

Initially: Aanya (Red), Ravi (Blue), Zoya (Green)After changes: Aanya becomes Yellow, Neera (Red) is added Only Neera has house "Red", so count = 1

Q.9. Prof. Andrew wants to print the names of all students who belong to the **"Red" house**. Which of the following statements about how he should approach this task is correct?

A. Loop over the marks dictionary and filter by house
B. Loop over the students list and check the "house" key in each dictionary
C. Use the key "Red" directly on the students list
D. Use the values() method on the list to get all houses and then print names

Since `students` is a list of dictionaries, you must iterate through each dictionary in the list.
Then check if the value of the `"house"` key equals `"Red"` and print the corresponding `"name"`.

Q.10. If Prof. Andrew wants to find out **how many students scored above 90 in the first subject,** what is the correct approach?

A. Check if marks[name][1] > 90 for all students
B. Loop through the students list and check "marks" key in each dictionary
C. Loop through the marks dictionary and check if the **first element** of the list is greater than 90
D. Sort the marks dictionary by name before checking scores

The marks are stored in a dictionary where each value is a list of marks.
To check the first subject (index 0) for all students, iterate through `marks.values()` and check if `mark_list[0] > 90`.

Bubble Sort is a simple sorting algorithm that works by **repeatedly comparing** adjacent elements in a list and **swapping them** if they are in the wrong order.

Imagine arranging numbers from smallest to largest. In each pass through the list, the largest number **"bubbles up"** to its correct position at the end. This process continues until the entire list is sorted.

```
numbers = [5, 1, 4, 2, 8]
```

- Compare 5 and 1 → swap
- Compare 5 and 4 → swap
- Compare 5 and 2 → swap
- Compare 5 and 8 → no swap

After the first pass, 8 has reached its correct place. More passes sort the rest.

Why is it called Bubble Sort?
Because with each pass, the **largest unsorted element "bubbles" to the top,** like a bubble rising in water.

Q.11. What will be the output after the **first complete pass** of bubble sort on the following list?

```
arr = [9, 4, 7, 3, 2]
```

A. [4, 7, 3, 2, 9]
B. [9, 7, 4, 3, 2]
C. [4, 3, 2, 7, 9]
D. [9, 4, 7, 3, 2]

@Exam Buddy

In the first pass of bubble sort, the largest element "bubbles up" to the end:

- Compare 9 and 4 → swap → [4, 9, 7, 3, 2]
- Compare 9 and 7 → swap → [4, 7, 9, 3, 2]
- Compare 9 and 3 → swap → [4, 7, 3, 9, 2]
- Compare 9 and 2 → swap → [4, 7, 3, 2, 9]

Q.12. How many total **swaps** will happen to fully sort the list below using bubble sort?

```
arr = [3, 2, 1]
```

A. 1
B. 2
C. 3
D. 0

In bubble sort, each pass moves the largest unsorted element to its correct position through adjacent swaps. For the list [3, 2, 1], the first pass requires 2 swaps to move 3 to the end, resulting in [2, 1, 3]. The second pass requires 1 swap to fix 2 and 1, resulting in [1, 2, 3], making a total of 3 swaps.

Q.13. Consider this bubble sort implementation:

```
arr = [5, 1, 4, 2, 8]
for i in range(len(arr)):
    for j in range(len(arr)-1):
        if arr[j] > arr[j+1]:
            arr[j], arr[j+1] = arr[j+1], arr[j]
print(arr)
```

What is the **output?**

A. [1, 2, 4, 5, 8]
B. [1, 4, 2, 5, 8]
C. [1, 2, 4, 8, 5]
D. Infinite loop

@Exam Buddy

The standard bubble sort implementation will completely sort the array.
After all passes, the list `[5, 1, 4, 2, 8]` becomes fully sorted in ascending order.

Q.14. What is the minimum number of **passes** required to sort this list using bubble sort?

```
arr = [1, 2, 3, 4, 5]
```

A. 0
B. 1

C. 4
D. 5

Even though the list is already sorted, bubble sort still requires one complete pass to verify no swaps are needed.
During this pass, no swaps occur, and the algorithm can terminate early if optimized, but the basic implementation runs at least one full pass.

Q.15. In which of the following cases will **bubble sort perform the maximum number of comparisons?**

A. Already sorted list
B. List sorted in reverse order
C. List with all elements the same
D. List with one element

Bubble sort performs the same number of comparisons regardless of the data (always $O(n^2)$ comparisons in basic implementation).
However, it performs the **maximum number of swaps** when the list is in reverse order, as every comparison results in a swap.

Q.16. What will be the output of the following code?

```
arr = [2, 3, 1, 5, 4]
for i in range(len(arr)-1):
    for j in range(len(arr)-1-i):
        if arr[j] < arr[j+1]:
            arr[j], arr[j+1] = arr[j+1], arr[j]
print(arr)
```

A. [5, 4, 3, 2, 1]
B. [4, 5, 3, 2, 1]
C. [5, 3, 4, 2, 1]
D. [2, 3, 1, 5, 4]

This is a **descending order** bubble sort (swaps when `arr[j] < arr[j+1]`).
The algorithm will completely sort the list `[2, 3, 1, 5, 4]` into descending order `[5, 4, 3, 2, 1]`.

Q.17. Which of the following would improve bubble sort's performance on already sorted lists?

A. Increasing the number of passes
B. Using a flag to break early if no swaps occur
C. Always sorting in reverse
D. Swapping even if elements are equal

This optimization checks if any swaps were made during a pass.
If no swaps occur, the list is already sorted, and the algorithm terminates early, saving unnecessary passes.

@Exam Buddy

Q.18. You have a list:

arr = [10, 20, 30, 40, 25]

After how many **passes** will 40 reach its correct position using bubble sort?

A. 1
B. 2
C. 3
D. 4

In bubble sort, the largest element "bubbles up" to its correct position in the first pass.
Here, 40 is already the largest and at the end, so it reaches its correct position after the **first pass**.

Q.19. What will be the final output of the following variant of bubble sort?

```
arr = [6, 3, 5, 2, 1]
n = len(arr)
for i in range(n):
    for j in range(i+1, n):
        if arr[i] > arr[j]:
            arr[i], arr[j] = arr[j], arr[i]
print(arr)
```

A. [1, 2, 3, 5, 6]
B. [2, 1, 3, 5, 6]
C. [6, 5, 3, 2, 1]
D. [1, 3, 2, 5, 6]

For the first code, it performs selection sort by comparing each element with all subsequent ones, eventually sorting `[6, 3, 5, 2, 1]` into `[1, 2, 3, 5, 6]`.

For the second code, the swapped flag detects the list `[1, 2, 3, 4, 5]` is already sorted after the first pass (i=0), then breaks. The loop runs once, so the final value of `i` printed is **1**.

Thus, the answers are `[1, 2, 3, 5, 6]` and `1` respectively.

Prof. Andrew wanted to make his Bubble Sort more efficient by adding a swapped flag to **break early** if no swaps happen during a pass.

Here's the code:

```python
arr = [1, 2, 3, 4, 5]
n = len(arr)

for i in range(n):
    swapped = False
    for j in range(n - 1 - i):
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]
            swapped = True
    if not swapped:
        break
print(i)
```

Q.20. What will be the output?

A. 0
B. 1
C. 4
D. 5

In Python, when the loop breaks after `i=0`, the current value of `i` is **0** when printed.
The `for` loop does not increment `i` before breaking when `swapped` is `False` during the first iteration.

It was the day before the annual sports day at college. Prof. Andrew was helping students line up for the march past, but he had a problem. The students were standing in random order based on their heights, and he wanted them arranged from **shortest to tallest.**

Instead of asking them all to sort themselves, Prof. Andrew used a simple strategy — something computers also use. It's called **Selection Sort.**

What is Selection Sort?

Selection Sort is a sorting algorithm where we select the smallest (or largest) item from the unsorted part of the list and put it at the correct position in the sorted part — step by step.

It's like arranging students in height order one at a time.

Let's say these are the student heights:

heights = [165, 150, 172, 160]

Step 1:

- Look through the entire list to find the **shortest student** (150).
- Swap it with the **first** student in line.
- Now the line looks like: [150, 165, 172, 160]

Step 2:

- From the remaining (unsorted) list [165, 172, 160], find the shortest → 160.
- Swap it with the second position.
- Now: [150, 160, 172, 165]

Step 3:

- From [172, 165], find the smallest → 165.
- Swap with third position → [150, 160, 165, 172]

Now the list is fully sorted!

@Exam Buddy

Q.21. Which of the following best describes the core idea of selection sort?

A. Swap every adjacent pair that is out of order.
B. Select the largest element and place it at the beginning.
C. Select the smallest element and place it at the correct position in each pass.
D. Randomly shuffle until the list is sorted.

In selection sort, we find the minimum element from the unsorted portion in each pass. We then swap it with the element at the current starting position of the unsorted part.

Q.22. How many swaps are made in total when sorting [4, 3, 2, 1] using selection sort?

A. 6

B. 3
C. 4
D. 2

- Pass 1: Min is 1 at index 3, swap with index 0 → [1, 3, 2, 4] (1 swap)
- Pass 2: Min in unsorted part is 2 at index 2, swap with index 1 → [1, 2, 3, 4] (2 swaps)
- Pass 3: Already sorted, no swap needed

Q.23. What will be the output of the following code?

```
arr = [5, 1, 4, 2]
for i in range(len(arr)):
    min_idx = i
    for j in range(i+1, len(arr)):
        if arr[j] < arr[min_idx]:
            min_idx = j
    arr[i], arr[min_idx] = arr[min_idx], arr[i]
print(arr)
```

A. [1, 2, 4, 5]
B. [5, 4, 2, 1]
C. [1, 4, 2, 5]
D. [1, 2, 5, 4]

Buddy

This is a standard selection sort implementation.
It finds the minimum element in the unsorted portion and swaps it to the current position, eventually sorting `[5, 1, 4, 2]` into `[1, 2, 4, 5]`.

Q.24. Which one of the following is **true** about the number of comparisons in selection sort?

A. It depends on whether the list is sorted or not
B. It varies based on data size and value

C. It always performs the same number of comparisons
D. It stops early if no swap is needed

Selection sort performs exactly n(n-1)/2 comparisons regardless of the input order. The algorithm always compares every element in the unsorted portion with the current minimum, making the comparison count fixed for a given input size.

Q.25. What happens if all elements in the list are already equal?

A. Selection sort performs no swaps or comparisons
B. Selection sort skips sorting
C. It performs comparisons but makes zero swaps
D. It performs swaps with random elements

Selection sort will still go through all its comparison steps to find the minimum element. However, since all elements are equal, the minimum index will never change from `i`, resulting in **zero swaps**.

@Exam Buddy

Q.26. What is the **output after the second pass** of selection sort on this list?

arr = [9, 3, 7, 1, 5]

A. [3, 1, 7, 9, 5]
B. [1, 3, 7, 5, 9]
C. [1, 3, 7, 9, 5]
D. [1, 3, 5, 7, 9]

- **Pass 1**: Find min=1 at index 3, swap with index 0 → [1, 3, 7, 9, 5]
- **Pass 2**: Find min in unsorted part [3,7,9,5] → min=3 at index 1, already in place → no swap → [1, 3, 7, 9, 5]

Q.27.

```
arr = [5, 1, 4, 2]
for i in range(len(arr)):
    min_idx = i
    for j in range(i+1, len(arr)):
        if arr[j] < arr[min_idx]:
            min_idx = j
    arr[i], arr[min_idx] = arr[min_idx], arr[i]
print(arr)
```

What is wrong with this code?

A. Nothing, it works fine
B. It doesn't place the smallest element correctly
C. It overwrites values and loses elements
D. It misses the last element in sorting

This is a standard, correct implementation of selection sort.
It will properly sort the array `[5, 1, 4, 2]` into `[1, 2, 4, 5]` without errors.

Q.28.

```
arr = [6, 3, 8, 5]
for i in range(len(arr) - 1):
    min_idx = i
    for j in range(i + 1, len(arr) - 1):
        if arr[j] < arr[min_idx]:
            min_idx = j
    arr[i], arr[min_idx] = arr[min_idx], arr[i]
print(arr)
```

What's the problem here?

A. Inner loop misses comparing the last element
B. It swaps elements incorrectly

C. Outer loop runs too many times
D. List becomes reversed

The inner loop `range(i + 1, len(arr) - 1)` stops one index before the end. This means the last element is never considered as a potential minimum, potentially leaving it unsorted.

Q.29.

```
arr = [9, 7, 5, 3]
for i in range(len(arr)):
    min_idx = 0
    for j in range(i + 1, len(arr)):
        if arr[j] < arr[min_idx]:
            min_idx = j
    arr[i], arr[min_idx] = arr[min_idx], arr[i]
print(arr)
```

What is the mistake here?

A. The code doesn't swap anything
B. The wrong element is selected as the minimum
C. min_idx is not reset correctly
D. The loop condition is wrong

The line `min_idx = 0` should be `min_idx = i` at the start of each outer loop iteration. By always setting `min_idx = 0`, the algorithm incorrectly compares elements with the first position instead of the current unsorted starting position.

Q.30.

```
arr = [1, 2, 3, 4]
for i in range(len(arr)):
    min_idx = i
    for j in range(i + 1, len(arr)):
        if arr[j] < arr[min_idx]:
            min_idx = j
    arr[i], arr[min_idx] = arr[min_idx], arr[i]
print(arr)
```

This code runs correctly, but what small optimization could improve performance?

A. Remove the outer loop
B. Break if the list is sorted
C. Check if min_idx != i before swapping
D. Change selection sort to bubble sort

This optimization avoids unnecessary swaps when the current element is already the minimum.
If `min_idx == i`, no swap is needed, saving the swap operation for that iteration.

@Exam Buddy

Week 7 Assignment

Refer to the code discussed in the "Joy of Computing using Python" lecture on implementing the Snake and Ladder game.

Ankan and Dilrez are playing a classic game of Snake and Ladder –but this time, it's implemented in Python! The game logic is based on the version you've seen in the lecture, where snakes pull you down and ladders push you up. The game board ranges from position 1 to 100.

The snake and ladder positions are stored in **dictionaries**. For instance:
snakes = {16: 6, 98: 78}
ladders = {1: 38, 80: 100}

Each player begins at position 0. In each turn, a player rolls a die (using random.randint(1, 6)) and moves forward. If they land on the bottom of a ladder, they climb up; if they land on a snake's mouth, they slide down. The game alternates between Ankan and Dilrez using a turn variable, and ends when any player reaches exactly 100.

Two key functions, check_ladder(position) and check_snake(position), determine whether the player hits a ladder or a snake after moving.

A function reached_end(position) is used to check if a player has reached position 100. A separate function is also written to open and display an image of the board using the **PIL** library.

Q.1. In the function play(), what is the significance of the variable turn?

A. It tracks the number of dice rolls
B. It ensures alternating turns between the two players
C. It ends the game after a fixed number of turns
D. It stores the winning score

The `turn` variable switches between 0 and 1 to alternate player turns.
This allows each player to roll the dice sequentially during the game.

Q.2. What happens in the code when a player rolls a number that moves them past position 100?

A. The game ends and they win
B. The player's position resets to 0
C. The player stays in the same position (doesn't move)
D. The player is forced to roll again

The code checks if `new_position > 100` and if so, doesn't update the position.
This prevents players from moving beyond the final position 100 on the board.

Q.3. What is the purpose of the following lines in the play() function?

```
if c == 0:
    print(p1_name, 'scored', pp1)
    print(p2_name, 'scored', pp2)
    print('Quitting the game. Thanks for playing.')
    break
```

A. To restart the game
B. To display final scores and exit if the player chooses not to continue
C. To check if the player reached a snake
D. To decide the winner

The condition checks if the user entered '0' to quit.
If so, it prints the current scores and breaks out of the game loop, ending the program.

@Exam Buddy

Q.4. If Ankan is at position 47 and the snakes dictionary contains 47:26, what will happen immediately after his dice roll moves him to 47?

A. He will stay at 47
B. He will move to 100
C. He will go to 26
D. The game will end

When a player lands on position 47, the code checks if it exists as a key in the `snakes` dictionary.
Since `snakes[47] = 26`, the player slides down to position 26.

Q.5. What does the function show_board() attempt to do?

A. Display current scores
B. Roll the dice for both players
C. Open and display the game board image
D. Initialize the snake and ladder dictionaries

The `show_board()` function uses `Image.open()` and `img.show()` to display an image file.
This visually shows the snakes and ladders game board to the players.

Q.6. Which of these functions is called after a player moves and before their position is printed?

A. show_board()
B. play()
C. check_ladder() and check_snake()
D. reached_end() only

After moving to a new position, the code calls both functions to check for ladders or snakes at that position.
These functions adjust the position if needed before it's displayed to the player.

@Exam Buddy

Q.7. How many times is the line random.randint(1, 6) evaluated in one full round of both players' turns?

A. Once
B. Twice
C. Four times
D. Depends on the player positions

Each player rolls the dice once per round, so `random.randint(1, 6)` is called once for Player 1 and once for Player 2.
This gives two dice rolls per complete round of both players' turns.

Q.8. Which part of the code ensures that the game stops immediately when a player reaches exactly 100?

A. if pp1 > end:
B. if pp1 == 100:
C. if reached_end(pp1):
D. if check_ladder(pp1):

The `reached_end()` function returns `True` when the position equals `end` (which is 100). This condition immediately breaks the game loop and declares the current player as the winner.

Consider the following ladders dictionary (in the format bottom: top): ladders = {1: 38, 4: 14, 9: 31, 21: 42, 28: 84, 36: 44, 51: 67, 71: 91, 80: 100}

Q.9. If a player is on **27**, rolls a **1**, and lands on **28**, what will happen next?

A. The player will stay on 28 because it's not a ladder base
B. The player climbs the ladder to 84
C. The player must roll again because 28 is a special position
D. The code throws an error because 28 is not handled explicitly

```
while True:
    if turn % 2 == 0:
        print(f"\n{p1_name}'s turn")
        ...
```

The `ladders` dictionary contains `28: 84`, so when the player lands on position 28, `check_ladder()` will return 84.
The player's position will be updated from 28 to 84 due to climbing the ladder.

Q.10. Why is the condition turn % 2 == 0 used in this part of the code?

A. To check if the game has reached the second round
B. To make sure player 2 only plays after player 1 wins

C. To alternate turns between the two players using the value of turn

D. To skip the dice roll for even-numbered turns

When `turn % 2 == 0`, it's Player 1's turn; when false, it's Player 2's turn. Incrementing `turn` after each player's move ensures the turns alternate properly throughout the game.

Dilrez, the ever-curious adventurer, had a habit of collecting strange and shiny things during her journeys.

One day, the village elder gifted her **two magical containers** — one called a **Backpack** and the other a **Pouch**.

The elder explained:

“Dilrez, these are no ordinary containers.

The **Backpack** holds items in a fixed order, and once packed, nothing can be changed.

The **Pouch**, on the other hand, only keeps unique items and doesn’t care about order.”

Dilrez was intrigued.

Backpack = Tuple

Dilrez packed it like this:

backpack = ("map", "compass", "torch")

He checked backpack[0] → 'map'.

But when he tried:

backpack[0] = "magic map"

He got an error!

Tuples are **ordered, immutable**, and allow **duplicates**.
Syntax: () (round brackets)

Pouch = Set

Dilrez filled his pouch:

pouch = {"coin", "potion", "coin"}

He expected 3 items, but got:

{'coin', 'potion'}

He added and removed items freely:

pouch.add("gem")

pouch.remove("coin")

Sets are **unordered, mutable**, and store **unique** items.
Syntax: {} (curly braces)

Q.11. What will be the output of the following code?

```
backpack = ("map", "compass", "torch")
print(backpack[1])
```

A. "map"
B. "torch"
C. "compass"
D. IndexError

Tuple indices start at 0, so `backpack[1]` accesses the second element.
The elements are: index 0 = "map", index 1 = "compass", index 2 = "torch"

Q.12. What will happen when Dilrez runs this code?

```
backpack = ("map", "compass", "torch")
backpack[0] = "magic map"
print(backpack)
```

A. Prints ("magic map", "compass", "torch")
B. Gives a TypeError
C. Prints original backpack
D. Gives a SyntaxError

Tuples are immutable in Python, meaning their elements cannot be changed after creation.
Trying to assign a new value to `backpack[0]` will raise a `TypeError: 'tuple' object does not support item assignment`.

Q.13. What will be the output?

```
pouch = {"coin", "potion", "coin"}
print(pouch)
```

Select all that apply:

A. The output will contain only unique elements
B. The output may appear in any order
C. It will raise a syntax error due to duplicate elements
D. The output will be a list with 2 elements

A set automatically removes duplicates, so `"coin"` appears only once.
Sets are unordered, so the print order may vary each time the code runs.

Q.14. What will be the final contents of the pouch?

```
pouch = {"potion"}
pouch.add("gem")
pouch.remove("potion")
print(pouch)
```

A. {'potion', 'gem'}
B. {'gem'}
C. ['gem']
D. Error due to remove()

The set starts as `{"potion"}`, then `"gem"` is added, making it `{"potion", "gem"}`. The `.remove("potion")` successfully removes "potion", leaving only `{"gem"}` in the set.

Q.15. What is the type of x in the following code?

```
x = ("compass")
print(type(x))
```

A. < class 'tuple' >
B. < class 'list' >
C. < class 'str' >
D. < class 'set' >

A single element in parentheses without a trailing comma is treated as a string, not a tuple. To create a single-element tuple, the syntax should be `x = ("compass",)`.

Q.16. Which of the following statements is **true**?

A. Tuples can’t store duplicate items
B. Sets maintain the order of elements
C. Sets automatically remove duplicate items
D. Tuples can be changed after creation

This is a fundamental property of sets in Python.
When you create a set with duplicate elements, it automatically removes all duplicates, keeping only unique items.

Q.17. What will this code output?

```
backpack = ("map", "map", "torch")
pouch = {"map", "map", "torch"}
print(len(backpack), len(pouch))
```

A. 3 3
B. 2 2
C. 3 2
D. 2 3

backpack is a tuple, so duplicates are kept, giving length 3.
pouch is a set, so duplicates are removed, giving length 2.
The print outputs 3 2 as the lengths of backpack and pouch respectively.

@Exam Buddy

Q.18. Dilrez tries to update a tuple. Find and fix the error:

```
gear = ("rope", "hook", "lamp")
gear[1] = "magic hook"
print(gear)
```

What's the issue?

A. Tuple index out of range
B. Syntax error in tuple
C. Tuples are immutable
D. Need to use append()

Tuples cannot be modified after creation, so assigning a new value to `gear[1]` is not allowed.
To "update" a tuple, you would need to create a new tuple with the desired changes.

Q.19. What will be the output of the following code?

```
items = ["map", "torch", "torch", "map"]
unique_items = set(items)
fixed_items = tuple(unique_items)

print(len(items), len(unique_items), fixed_items)
```

A. 4 2 ('map', 'torch')
B. 4 2 ('torch', 'map')
C. 2 2 ('map', 'torch')
D. 4 4 ('map', 'torch', 'torch', 'map')

- `len(items)` = 4 (original list with duplicates)
- `len(unique_items)` = 2 (set removes duplicates)
- `fixed_items` can be `('torch', 'map')` or `('map', 'torch')` since sets are unordered

Both A and B are valid outputs because the tuple order depends on the set's internal ordering, which is arbitrary.

Q.20. Dilrez does the following:

```
backpack = ("map", "torch")
pouch = {"torch", "map"}
supply_list = ["map", "torch"]
```

Which of the following is true based on this setup?

A. All three containers contain the same items and in the same order
B. Only the pouch automatically removes duplicates
C. backpack[1] and supply_list[1] give the same result
D. Sets can be indexed just like lists and tuples

Only the pouch automatically removes duplicates – Sets remove duplicates, while tuples and lists preserve all elements including duplicates.

C. backpack[1] and supply_list[1] give the same result – Both `backpack[1]` and `supply_list[1]` return `"torch"` since indexing works the same way for tuples and lists.

Himmat was given a task: Find the **factorial** of a number.

He read that:

The factorial of a number n is written as n!
It means:

n! = n × (n-1) × (n-2) × ... × 1

So, for example:

4! = 4 × 3 × 2 × 1 = 24

But instead of doing it manually, Himmat decided to write a **recursive function** — a function that calls **itself**.

```
def factorial(n):
  if n == 0:
    return 1
  return n * factorial(n - 1)
```

Q.21. Why does the code return 1 when n == 0?

A. Because 0 × anything is 0
B. To stop the recursion at some point
C. By mistake; it should return 0
D. It helps to calculate square of the number

This is the **base case** that prevents infinite recursion.
In mathematics, 0! is defined as 1, which also correctly terminates the recursive multiplication chain.

Q.22. What will be the return value of factorial(1)?

A. 0
B. 1
C. It will call itself infinitely
D. 2

When `n = 1`, the function calculates:

- `factorial(1)` → `1 * factorial(0)`
- `factorial(0)` returns `1` (base case)
- So `1 * 1 = 1`

@Exam Buddy

Q.23. Identify the error in the following code:

```
def factorial(n):
    return n * factorial(n - 1)

print(factorial(3))
```

A. It will return 6
B. No base case defined, will result in infinite recursion
C. n is not decremented
D. Syntax error

The function calls itself repeatedly with `n-1` but has no condition to stop when `n` reaches 0.
This causes infinite recursion until a `RecursionError` occurs.

Q.24. What does the line n! = n × (n-1) × (n-2) × ... × 1 represent in the comprehension?

A. An infinite series
B. A summation of natural numbers
C. A recursive definition of factorial
D. A non-recursive mathematical shortcut

The line `n! = n × (n-1) × (n-2) × ... × 1` represents the **iterative/expanded mathematical definition** of factorial.
It shows the direct multiplication sequence rather than a recursive self-reference, making it a non-recursive way to understand the calculation.

Q.25.

```
def factorial(n):
    if n == 0:
        return 1
    else:
        return factorial(n - 1) * n
```

How many recursive calls are made when he executes factorial(4)?

A. 3
B. 4
C. 5
D. Infinite

The recursive calls are:

- `factorial(4)` calls `factorial(3)`
- `factorial(3)` calls `factorial(2)`
- `factorial(2)` calls `factorial(1)`
- `factorial(1)` calls `factorial(0)` (base case)

That's **4** recursive calls before reaching the base case.

Q.26. Himmat accidentally writes:

```
def factorial(n):
    if n == 1:
        return 1
    return n * factorial(n - 1)
```

What will happen if he calls factorial(0)?

A. Returns 1
B. Returns 0
C. Throws a recursion error
D. Returns -1

@Exam Buddy

When `factorial(0)` is called, the base case `n == 1` is false, so it calls `factorial(-1)`.
This continues with `factorial(-2)`, `factorial(-3)`, etc., causing infinite recursion until a `RecursionError` occurs.

Q.27. What would be a correct way to **make Himmat's function handle negative numbers safely?**

A. Add return -1 for all inputs
B. Add a base case if n < 0: return "Invalid input"
C. Remove recursion altogether
D. Multiply by -1 inside the function

This would check for negative inputs at the start and return an error message.
It prevents infinite recursion by handling invalid values before the recursive call.

Q.28. If Himmat's function is modified as:

```
def factorial(n):
    if n == 0:
        return 1
    return factorial(n) * n
```

What happens when factorial(3) is called?

A. Returns 6
B. Returns 0
C. Infinite recursion
D. Error due to syntax

The function calls `factorial(n)` with the same value of `n` repeatedly, never reaching the base case.
This creates infinite recursion since `n` never decreases, eventually causing a `RecursionError`.

@Exam Buddy

Q.29. Himmat runs this code to reverse a string:

```
def reverse_string(s):
    if len(s) <= 1:
        return s
    return reverse_string(s[1:]) + s[0]

print(reverse_string("a"))
```

What will be the output?

A. "a"
B. ""

C. Error (index out of range)
D. "aa"

The input string has length 1, so the base case `len(s) <= 1` is triggered.
The function returns the string `"a"` immediately without any recursive calls.

Q.30. Himmat wants to reverse the word "race" using recursion. He writes a function that checks whether the string is non-empty as the base case.

```
def reverse_string(s):
    if s:
        return reverse_string(s[1:]) + s[0]
    else:
        return ""
reverse_string("race")
```

What will be the output of reverse_string("race")?

A. "race"
B. "ecar"
C. "rcae"
D. Error due to slicing empty string

- `reverse_string("race")` → `reverse_string("ace") + "r"`
- `reverse_string("ace")` → `reverse_string("ce") + "a"`
- `reverse_string("ce")` → `reverse_string("e") + "c"`
- `reverse_string("e")` → `reverse_string("") + "e"`
- `reverse_string("")` returns `""`

Now building back: `"" + "e" = "e"`, then `"e" + "c" = "ec"`, then `"ec" + "a" = "eca"`, then `"eca" + "r" = "ecar"`

Week 8 Assignment

An anagram is a word formed by rearranging the letters of another word. For example, the word "**listen**" can be turned into "**silent**". Both words use the exact same letters, just in a different order.

Anagrams are often used in word puzzles and games. Some fun examples include:

- **evil → vile**
- **dusty → study**
- **save → vase**

To check if two words are anagrams, you can sort their letters and see if they match.

Q.1. Which of the following is an example of an anagram?

A. cat → dog
B. loop → pool
C. red → reed
D. cup → mug

An anagram rearranges the letters of one word to form another word using all the original letters exactly once.
"loop" rearranged becomes "pool" — same letters, different order.

@Exam Buddy

Q.2. What will be the output of the following code?

```
def are_anagrams(w1, w2):
    return sorted(w1) == sorted(w2)

print(are_anagrams("Listen", "Silent"))
```

A. True
B. False
C. Error: cannot compare lists
D. It depends on the length of the words

he function compares `sorted("Listen")` and `sorted("Silent")` which are:

- `['L', 'e', 'i', 'n', 's', 't']` vs `['S', 'e', 'i', 'l', 'n', 't']`
  The capital 'L' and 'S' have different ASCII values, so the sorted lists are not equal.

Q.3. What will be the output of the following code?

```
def are_anagrams(w1, w2):
    w1 = w1.replace(" ", "").lower()
    w2 = w2.replace(" ", "").lower()
    return sorted(w1) == sorted(w2)

print(are_anagrams("Dormitory", "Dirty room"))
```

A. True
B. False
C. False because the strings are of different original lengths
D. Depends on whether the input has punctuation

The function removes spaces and converts both strings to lowercase before sorting.
The sorted versions of "dormitory" and "dirtyroom" produce identical character lists.
Therefore, the function correctly identifies them as anagrams and returns **True**.

Q.4. How many **distinct anagrams** can be formed using all the letters of the word **“LEVEL”**?

A. 60
B. 120
C. 30
D. 20

The word "LEVEL" has 5 letters with repetitions: L appears 2 times, E appears 2 times, V appears once.
The formula for distinct anagrams is:

5! / (2! × 2! × 1!) = 120 / (2 × 2) = **30**

Q.5. What will be the output of the following code?

```
words = ["listen", "silent", "enlist", "google", "gogole"]

anagrams = []
for i in range(len(words)):
    for j in range(i + 1, len(words)):
        if sorted(words[i]) == sorted(words[j]):
            anagrams.append((words[i], words[j]))

print(anagrams)
```

A. [('listen', 'silent'), ('listen', 'enlist'), ('silent', 'enlist'), ('google', 'gogole')]
B. [('listen', 'silent'), ('silent', 'enlist'), ('google', 'gogole')]
C. [('listen', 'silent'), ('listen', 'enlist'), ('google', 'gogole')]
D. [('listen', 'silent'), ('listen', 'enlist'), ('silent', 'enlist')]

The nested loop compares each word with every subsequent word in the list using sorted character comparison.
It finds all anagram pairs: listen-silent, listen-enlist, silent-enlist, and google-goggle.
All four matching pairs are collected and printed in the anagrams list.

Q.6. The following function is intended to check if two words are anagrams.

However, it **fails for some valid inputs**. Identify the bug.

```
def is_anagram(w1, w2):
    return sorted(w1) == sorted(w2)
```

What is the issue with this code?

A. It doesn't handle uppercase and lowercase letters correctly
B. sorted() can't be used on strings
C. The return statement compares the memory addresses

D. It requires the words to be of equal length

The function compares strings case-sensitively, so "Listen" and "Silent" would return False.
It should convert both strings to the same case (e.g., lowercase) before sorting and comparing.

Q.7. Which of the following must be true for two strings to be anagrams?

A. Both strings must be in the dictionary
B. Both strings must be the same length and have identical character counts
C. Both strings must have the same starting letter
D. Both strings must have the same number of vowels

Anagrams must use exactly the same letters with the same frequency.
This means they must have equal length and identical character counts, regardless of letter order.

Q.8. You are checking whether "abcde" and "edcba " (note the space at the end) are anagrams. Which of the following **best describes** why a naive checker would fail?

A. Because they have different lengths
B. Because the space has ASCII value 32
C. Because sorted lists don't match
D. All of the above

They have different lengths (5 vs 6 characters). The space has ASCII value 32, affecting sorting. The sorted lists are `['a','b','c','d','e']` vs `[' ','a','b','c','d','e']`

Q.9. What will be the output of the following code?

```
def anagram_checker(word):
    return word == word[::-1]

print(anagram_checker("madam"))
```

A. True
B. False
C. Error: invalid slicing syntax
D. True, because "madam" is an anagram of "madam

The function actually checks if the word is a palindrome, not an anagram.
`word[::-1]` reverses the string, and "madam" reversed is still "madam".
So the comparison returns `True` for this specific case.

@Exam

Q.10. What will be the output of the following code?

```
def ascii_sum(word):
    total = 0
    for char in word:
        total += ord(char)
    return total

print(ascii_sum("abc") == ascii_sum("acb"))
```

A. True
B. False
C. Error: ord() cannot be used in loop
D. False, because the order of characters matters in ord()

The ASCII sum of a word depends only on its characters, not their order.
Both "abc" and "acb" contain the same letters 'a', 'b', and 'c'.
Therefore, their ASCII sums are equal, and the function returns `True`.

A player randomly guesses a number from 1 to 10. The computer also picks a random number in the same range. If the guess is wrong, the score increases by 1. If it's correct, the score decreases by 1. This process repeats 10 times, and the score after each round is plotted on a graph.

```
import random
import matplotlib.pyplot as plt

counts = []
count = 0

for i in range(10):
    guess = random.randint(1, 10)
    pick = random.randint(1, 10)

    if guess != pick:
        count += 1
    else:
        count -= 1

    counts.append(count)

plt.plot(counts)
plt.show()
```

Q.11. What is the range of numbers used for guessing in each round?

A. 1 to 5
B. 0 to 9
C. 1 to 10
D. 0 to 10

The code uses `random.randint(1, 10)` for both `guess` and `pick`.
This function includes both endpoints, generating random integers from 1 to 10 inclusive.

Q.12. What happens to the score if the player's guess matches the computer's pick?

A. It increases by 1
B. It stays the same
C. It decreases by 1
D. It becomes 0

When `guess == pick`, the code executes the `else` branch: `count -= 1`.
This decreases the score by 1 when the player guesses correctly.

Q.13. How many values will be stored in the counts list after the loop finishes?

A. 0
B. 5
C. 9
D. 10

The loop runs 10 times (`for i in range(10)`).
In each iteration, `count` is updated and appended to the `counts` list.
So after the loop, `counts` contains exactly 10 values.

@Exam Buddy

Q.14. Which of the following best describes the purpose of the final plt.plot(counts) line?

A. It shows how many times the guess was correct
B. It prints the final score
C. It displays how the score changed over each round
D. It compares the guess and pick values directly

The `counts` list records the cumulative score after each round.
`plt.plot(counts)` creates a line graph showing how the score evolved over all 10 rounds.

Q.15. Under which of the following conditions will the plot of the counts list form a straight line of the form y = mx + c (either increasing or decreasing at a constant rate)?

A. The guess matches the pick in some random rounds
B. The guess never matches the pick throughout all 10 rounds
C. The count increases or decreases based on a sine function
D. The guess and pick values are both fixed at 5 in every round

If the guess never matches the pick, the code always executes `count += 1`.
This would make the score increase by exactly 1 each round, creating a straight line with constant slope.

Q.16. Which of the following changes will make the score decrease with every guess?

A. Replace count += 1 with count -= 1
B. Make guess == pick always true
C. Set count = 0 in every iteration
D. Remove counts.append(count)

This change would make the score decrease by 1 when the guess is wrong.
Since guesses are random and often incorrect, the score would consistently decrease across most rounds.

Q.17. What will happen if the line counts.append(count) is removed from the code?

A. The graph will still show the score progression
B. The code will crash due to a missing list
C. The counts list will remain empty, and the plot will be blank
D. The program will enter an infinite loop

Without `counts.append(count)`, no values are added to the `counts` list.
When `plt.plot(counts)` is called with an empty list, it will display a blank graph with

no data points.

Q.18. What will happen if random.randint(1, 10) is replaced with random.randint(10, 1)?

A. The guess will always be 10
B. The range will reverse and still work
C. A ValueError will be raised due to invalid range
D. The output will be a float instead of an int

The `random.randint(a, b)` function requires that `a <= b`.
If called as `random.randint(10, 1)` where 10 > 1, Python will raise a `ValueError`.

Q.19. How can you modify the code to simulate 100 guessing rounds instead of 10?

A. Change range(10) to range(100)
B. Replace random.randint(1, 10) with random.randint(1, 100)
C. Increase the value of count by 100
D. Run the code 10 times in a loop

The loop runs for the number of iterations specified in `range()`.
Changing `range(10)` to `range(100)` will make the loop execute 100 times instead of 10.

Q.20. How can you modify the code to plot both the guessed values and the picked values across rounds?

A. Use a bar chart to show both values in one plot
B. Store guess and pick values in separate lists and plot both using plt.plot()
C. Plot only the counts list as it includes both guess and pick
D. Multiply the guess and pick values and store them in one list

Create two empty lists `guesses` and `picks` before the loop.
Inside the loop, append each `guess` and `pick` to their respective lists.
Then use `plt.plot(guesses)` and `plt.plot(picks)` to display both lines on the same graph.

Detective Neel was walking down the street when he noticed a blurry poster of a missing dog. The colors were faded, and the image barely recognizable. Curious, he decided to help. He took a photo of the poster with his phone and went back to his computer.

Neel had recently learned about a Python library called **PIL**—short for **Python Imaging Library**. It helped programmers **open, view, resize**, and **enhance** images using simple code.

First, he wrote:

```
from PIL import Image
```

This allowed him to access the Image class from the PIL module. He then opened the dog image with:

```
img = Image.open("dog_poster.jpg")
```

With just one line, he loaded the photo into Python! To make it easier to analyze, Neel resized it:

```
img_resized = img.resize((300, 300))
```

He printed out the format and mode of the image:

```
print(img.format)  # e.g., JPEG
print(img.mode)    # e.g., RGB or L
```

Finally, Neel displayed the image on screen using:

```
img_resized.show()
```

Q.21. What does the PIL library allow users to do?

A. Send emails
B. Manipulate and view images
C. Write games
D. Draw graphs

PIL (Python Imaging Library) provides tools for opening, manipulating, and saving various image file formats.
It enables operations like resizing, cropping, filtering, and displaying images in Python.

Q.22. What was the purpose of Image.open("dog_poster.jpg")?

A. To download an image
B. To open the file in a browser
C. To load the image into Python
D. To delete the file

`Image.open()` reads the image file from disk and creates an Image object.
This allows the image data to be accessed and manipulated programmatically within Python.

Q.23. What does img.resize((300, 300)) do?

A. Crops the image
B. Zooms into the image
C. Resizes the image to 300x300 pixels
D. Rotates the image

The `resize()` method changes the dimensions of the image to the specified width and height.
It scales the entire image to fit within a 300x300 pixel area, potentially changing its aspect ratio.

Q.24. If img.mode prints "RGB", what does that mean?

A. The image has only black and white shades
B. The image uses Red, Green, and Blue channels
C. The image is rotated
D. The image is grayscale

"RGB" mode means the image stores color information using three channels: Red, Green, and Blue.
This is the standard color model for digital images and displays.

@Exam Buddy

Q.25. Which function is used to view the image on the screen?

A. img.open()
B. img.save()
C. img.show()
D. img.print()

The `show()` method opens the image in the system's default image viewer application.
This allows users to visually inspect the image directly from Python.

Q.26. What will be the output type of this code?

```
from PIL import Image
img = Image.open("dog_poster.jpg")
print(type(img))
```

A. < class 'str' >
B. < class 'PIL.JpegImagePlugin.JpegImageFile' >

C. < class 'Image' >
D. < class 'PIL.Image.Image' >

The `Image.open()` method returns an image object whose type depends on the file format.
For JPEG files like "dog_poster.jpg", it specifically returns a `JpegImageFile` instance from the JPEG plugin.
This is the actual class used to handle JPEG images in PIL, even though it inherits from `PIL.Image.Image`.

Q.27. What will the following code print?

```
img = Image.open("dog_poster.jpg")
img = img.resize((3, 3))
arr = np.array(img)
print(arr.shape)
```

A. (3, 3)
B. (3, 3, 3)
C. 9
D. Error, images cannot be converted to arrays

The image is resized to 3x3 pixels and converted to a NumPy array.
Since it's an RGB image, the array shape is (height, width, channels) → (3, 3, 3).

Q.28. If the original image shape is (4, 4, 3), what would be the shape of the compressed array after slicing arr[::2, ::2]?

A. (2, 2, 3)
B. (2, 2)
C. (4, 2, 3)
D. (2, 4, 3)

The slice `[::2, ::2]` selects every second row and every second column.
This reduces the 4×4 dimensions to 2×2 while preserving all 3 color channels.

Q.29. What does each arr[x][y] contain in a colored image loaded with NumPy?

A. A matrix
B. A string
C. A list of RGB values
D. A single grayscale value

In a colored image array, `arr[x][y]` represents a single pixel at position (x,y).
For an RGB image, this contains a list/tuple of three values: [Red, Green, Blue] intensities.

Q.30. Which of the following is a correct sequence to open, resize, and view an image?

```
img.resize((100, 100))
Image.open("poster.jpg")
img.show()
```

```
img = Image.open("poster.jpg")
img = img.resize((100, 100))
img.show()
```

```
Image.open("poster.jpg").show()
img = resize(img, (100, 100))
```

```
open("poster.jpg")
resize(img)
show(img)
```

```
img = Image.open("poster.jpg")
img = img.resize((100, 100))
img.show()
```

First open the image file and create an Image object.
Then resize the image to the desired dimensions.
Finally, display the modified image using the show() method.



@Exam Buddy

Week 9 Assignment

Many real-world systems can be understood better when modeled as networks. Think about social networks like Instagram, road maps connecting cities, biological pathways, or the structure of the

internet. Each of these can be represented using nodes (the entities) and edges (the relationships between them).

To work with such networks in Python, we use a library called NetworkX. This library helps us easily build and analyze networks using just a few lines of code.

Let's walk through a basic example and understand what each line does:

```
import networkx as nx
import matplotlib.pyplot as plt
```

We begin by importing the necessary libraries. networkx is used for creating and analyzing the graph, while matplotlib.pyplot helps us visualize it.

```
G = nx.Graph()
```

This line creates an empty undirected graph named G. Think of this as starting with a blank canvas where we'll add people (nodes) and their connections (edges).

```
G.add_node("Alice")
G.add_node("Bob")
```

We add two individual nodes to our graph: "Alice" and "Bob". A node can represent any entity : a person, a city, or even a webpage.

```
G.add_edge("Alice", "Bob")
```

This adds a connection between Alice and Bob. If either of these nodes didn't exist already, NetworkX would automatically add them first.

```
G.add_edges_from([("Bob", "Charlie"), ("Alice", "David")])
```

Here, we add multiple edges at once. This line connects Bob to Charlie and Alice to David. Again, if "Charlie" or "David" weren't added earlier, they will be automatically created.

```
nx.draw(G, with_labels=True)
plt.show()
```

This code is used for drawing the graph. nx.draw() creates a visual layout, and with_labels=True ensures that node names (like "Alice") are shown. plt.show() then displays the graph window.

```
print("Nodes:", G.nodes())
print("Edges:", G.edges())
print("Number of nodes:", G.number_of_nodes())
print("Number of edges:", G.number_of_edges())
```

These lines print basic information about the graph.

- G.nodes() gives a list of all nodes.
- G.edges() shows all the connections.
- G.number_of_nodes() counts how many unique entities are in the graph.
- G.number_of_edges() counts how many relationships exist between them.

This simple example shows how easy it is to model and explore networks using NetworkX. From here, you can move on to more complex tasks like finding the shortest path, checking connected components, or analyzing social structures.

Q.1. What will be the output of the following code?

```
import networkx as nx
G = nx.Graph()
G.add_edge("A", "B")
print("C" in G.nodes())
```

A. True
B. False
C. Error: "C" is not defined
D. None

The graph only contains nodes "A" and "B" from the added edge.
The check `"C" in G.nodes()` searches for node "C", which doesn't exist in the graph.

Q.2. Predict the output of the following code.

```
import networkx as nx
G = nx.Graph()
G.add_edges_from([("X", "Y"), ("Y", "Z")])
print(G.number_of_nodes())
```

A. 2
B. 3
C. 1
D. Error

The edges connect three distinct nodes: "X", "Y", and "Z".
`G.number_of_nodes()` counts all unique nodes in the graph, which is 3.

Q.3. What does G.add_edges_from([("A", "B"), ("B", "C")]) do?

A. Adds nodes only if they already exist
B. Adds edges and automatically creates any missing nodes
C. Only creates the nodes without connecting them
D. Adds weighted edges between existing nodes

NetworkX automatically creates any nodes that don't already exist when adding edges.
This command adds both edges A-B and B-C, creating nodes A, B, and C in the process.

Q.4. What is the output of the given code ?

```
import networkx as nx
G = nx.Graph()
G.add_node("Tom")
print(G.number_of_edges())
```

A. 1
B. 0

C. Error
D. Depends on the number of nodes

The code only adds a single node "Tom" to the graph.
No edges are added, so `G.number_of_edges()` returns 0.

Q.5. Which of the following is TRUE after executing only this line?

```
G = nx.Graph()
```

A. One empty node is created
B. A graph with no nodes or edges is created
C. A directed graph is created
D. A graph with one edge and no node is created

`nx.Graph()` initializes an empty undirected graph object.
It starts with zero nodes and zero edges until elements are explicitly added.

@Exam Buddy

Q.6. Why is plt.show() used after nx.draw(G, with_labels=True)?

A. To print the node names
B. To display the graph visualization
C. To calculate shortest path
D. To show node degrees

`nx.draw()` creates the graph plot in memory but doesn't display it.
`plt.show()` renders and displays the visualization window with the drawn graph.

Q.7. What happens if you add an edge like G.add_edge("A", "B") when "A" and "B" are not yet added?

A. Error: You must add nodes first
B. Only the edge is created, not the nodes
C. Nodes "A" and "B" are automatically added
D. Only "A" is added, "B" is ignored

The correct answer is: **Nodes "A" and "B" are automatically added**

When you call `G.add_edge("A", "B")` in NetworkX, both nodes "A" and "B" are automatically created and added to the graph if they don't already exist. You don't need to add nodes separately before adding edges.

Q.8. Predict the output

```
import networkx as nx
G = nx.Graph()
G.add_node("A")
G.add_node("B")
print(G.edges())
```

A. [('A', 'B')]
B. [ ]
C. ['A', 'B']
D. Error

The code adds two nodes "A" and "B" but no edges between them.
`G.edges()` returns a list of all edges in the graph, which is empty in this case.

Q.9. You are building a graph step-by-step using the following sequence of operations:

1. Create an empty graph using G = nx.Graph()
2. Add edge: G.add_edge("A", "B")
3. Add edge: G.add_edge("B", "C")
4. Add node: G.add_node("D")

How many nodes and edges will exist in the graph after these steps?

A. 4 nodes, 2 edges
B. 3 nodes, 2 edges
C. 3 nodes, 3 edges
D. 4 nodes, 3 edges

Step 2 adds edge A-B, creating nodes A and B automatically.
Step 3 adds edge B-C, creating node C and connecting it to B.
Step 4 adds node D separately, resulting in 4 nodes and 2 edges total.

Q.10. Analyse the code given below :

```
G = nx.Graph()
G.add_nodes_from(["A", "B", "C", "D"])
G.add_edges_from([("A", "B"), ("B", "C"), ("A", "C")])
nx.draw(G, with_labels=True)
plt.show()
```

Which of the following statements is most accurate?

A. Node D will not be visible in the graph because it has no edges
B. Node D will be visible but disconnected
C. Node D is automatically removed before drawing
D. The graph will throw an error because node D is isolated

`nx.draw()` displays all nodes in the graph regardless of whether they have edges. Node D will appear as an isolated point in the visualization, disconnected from other nodes.

Once you've created a graph using NetworkX by adding nodes and edges, the next step is to analyze its structure. NetworkX provides several built-in functions to help us understand how different parts of the network are connected.

Let's take this example:

```
import networkx as nx
G = nx.Graph()

G.add_edges_from([("Alice", "Bob"), ("Bob", "Charlie"), ("Charlie", "David"), ("Alice", "David")])
```

Degree of a Node

The degree of a node is the number of edges connected to it. For example:

```
print(G.degree("Bob"))
```

This will return 2 because Bob is connected to both Alice and Charlie.

Neighbors of a Node

To get all nodes connected to a particular node (its neighbors), we can use:

```
list(G.neighbors("Charlie"))
```

This will return ['Bob', 'David'] because Charlie is connected to both.

Checking for Paths

To check if there is a path between two nodes, we can use:

```
nx.has_path(G, "Alice", "Charlie")
```

This returns True because Alice → Bob → Charlie is a valid path.

Shortest Path

To find the shortest connection between two nodes:

```
nx.shortest_path(G, "Alice", "Charlie")
```

These tools are powerful in understanding how information flows through a network, how connected the network is, and how close different nodes are to each other.

Q.11. Given the following graph construction:

```
G = nx.Graph()
G.add_edges_from([("Alice", "Bob"), ("Bob", "Charlie"), ("Charlie", "David"), ("Alice", "David")])
```

What will be the output of G.degree("Charlie")?

A. 1
B. 2
C. 3
D. 4

Node "Charlie" is connected to "Bob" and "David" via edges.
The degree counts the number of edges connected to a node, which is 2 for "Charlie".

@Exam Buddy

Q.12. What is the correct way to find the neighbors of "Bob"?

A. G.neighbors("Bob")
B. list(G.neighbors("Bob"))
C. G.degree("Bob")
D. nx.neighbors(G, "Bob")

`G.neighbors("Bob")` returns an iterator over Bob's adjacent nodes.
Wrapping it with `list()` converts it to a visible list of neighbor nodes like `['Alice', 'Charlie']`.

Q.13.

```
G = nx.Graph()
G.add_edges_from([("Alice", "Bob"), ("Bob", "Charlie"), ("Charlie", "David"), ("Alice", "David")])
```

For this graph, what does nx.shortest_path(G, "Alice", "Charlie") return?

A. ['Alice', 'Charlie']

B. ['Alice', 'David', 'Charlie']
C. ['Alice', 'Bob', 'Charlie']
D. Both B and C are valid paths

The shortest path from Alice to Charlie has length 2.
Path A-B-C (Alice-Bob-Charlie) is shorter than A-D-C (Alice-David-Charlie), which has length 3.
`nx.shortest_path()` returns one shortest path, typically the first found.

Q.14. What will be the number of nodes after executing the following code?

```
edges = [("A", "B"), ("B", "C"), ("C", "D")]
G = nx.Graph()
G.add_edges_from(edges)
print(len(G.nodes()))
```

A. 2
B. 3
C. 4
D. Cannot be determined

@Exam Buddy

The edges connect four distinct nodes: A, B, C, and D.
`G.add_edges_from()` automatically creates all nodes mentioned in the edge list.
So `len(G.nodes())` returns 4.

Q.15. Consider the following code:

```
G = nx.Graph()
G.add_edges_from([("A", "B"), ("B", "C")])
degrees = {node: G.degree(node) for node in G.nodes()}
```

What will be the value of degrees["B"]?

A. 1
B. 2
C. 3

D. Error

Node "B" is connected to both "A" and "C" via edges.
The degree counts the number of connections, so `G.degree("B")` returns 2.

Q.16. Which of the following will correctly collect the neighbors of all nodes in a dictionary?

A. {n: G.neighbors(n) for n in G.nodes()}
B. {n: list(G.neighbors(n)) for n in G.nodes()}
C. {n: G.degree(n) for n in G.nodes()}
D. {n: nx.shortest_path(n) for n in G.nodes()}

`G.neighbors(n)` returns an iterator, which must be converted to a list to see the values.
This dictionary comprehension creates a mapping from each node to its list of adjacent neighbors.

Q.17. Given the graph:

```
G = nx.Graph()
G.add_edges_from([("A", "B"), ("B", "C")])
path = nx.shortest_path(G, "A", "C")
```

What is the value of path[1]?

A. "A"
B. "B"
C. "C"
D. Error

The shortest path from A to C is `['A', 'B', 'C']`.
Indexing `path[1]` accesses the second element of this list, which is node "B".

Q.18. What will be the output of the following code?

```
G = nx.Graph()
G.add_edges_from([("A", "B"), ("C", "D")])
pairs = [("A", "B"), ("A", "C")]
results = [nx.has_path(G, u, v) for u, v in pairs]
```

What is the value of results?

A. [True, True]
B. [False, False]
C. [True, False]
D. Error

`nx.has_path()` checks if there's any path between two nodes.
A-B are directly connected (True), but A and C are in disconnected components (False).

Q.19. What does the following code store in paths?

```
G = nx.Graph()
G.add_edges_from([("X", "Y"), ("Y", "Z")])
nodes = ["X", "Z"]
paths = {n: nx.shortest_path(G, "X", n) for n in nodes}
```

A. Paths from "X" to all other nodes
B. Error, cannot use dict with shortest_path
C. Shortest path from "X" to "X" and "X" to "Z"
D. Only the path from "X" to "Z"

The dictionary comprehension iterates over `nodes = ['X', 'Z']`.
It computes paths: `"X"→"X"` = `['X']` and `"X"→"Z"` = `['X','Y','Z']`.

Q.20. Given:

```
G = nx.Graph()
G.add_edges_from([("A", "B"), ("B", "C"), ("C", "D")])
high_deg = [n for n in G.nodes() if G.degree(n) > 1]
```

What is high_deg?

A. ["B", "C"]
B. ["A", "D"]
C. ["A", "B", "C"]
D. [ ]

Node degrees: A=1, B=2, C=2, D=1.
The list comprehension filters nodes with degree > 1, so only B and C qualify.

@Exam Buddy

© Exam Buddy → NPTEL- JULY 2025 Course Notes Gmail – nptelsolver@gmail.com



Week 10 Assignment

In digital image processing, quantization is a technique used to reduce the number of possible pixel intensity values. This is often done to compress an image or simplify its representation for analysis.

The following Python program demonstrates **intensity quantization** using the **Python Imaging Library** (PIL) and **NumPy.** The program loads a grayscale image, examines the intensity of each pixel, and replaces it with a new value based on which range it falls into. The intensity values are divided into 8 equal levels ranging from 0 to 255.

```python
from PIL import Image
import numpy

# Step 1: Load the image in grayscale mode
im = Image.open('images.jpg').convert('L')  # Ensure grayscale
pixelMap = im.load()

# Step 2: Create a new grayscale image
img = Image.new('L', im.size)
pixelNew = img.load()

# Step 3: Quantize
for i in range(img.size[0]):
    for j in range(img.size[1]):
        if 0 <= pixelMap[i, j] <= 31:
            pixelNew[i, j] = 0
        elif 32 <= pixelMap[i, j] <= 63:
            pixelNew[i, j] = 1
        elif 64 <= pixelMap[i, j] <= 95:
            pixelNew[i, j] = 2
        elif 96 <= pixelMap[i, j] <= 127:
            pixelNew[i, j] = 3
        elif 128 <= pixelMap[i, j] <= 159:
            pixelNew[i, j] = 4
        elif 160 <= pixelMap[i, j] <= 191:
            pixelNew[i, j] = 5
        elif 192 <= pixelMap[i, j] <= 223:
            pixelNew[i, j] = 6
        elif 224 <= pixelMap[i, j] <= 255:
            pixelNew[i, j] = 7


img.save('lena_2.jpg')
J = numpy.asarray(Image.open('lena_2.jpg'))
```

In the code above:

- The image is loaded and converted to grayscale using .convert('L').
- The image is processed pixel by pixel. The comparison is directly made using pixelMap[i, j].
- The if-elif structure checks which of the eight 32-level bins the pixel belongs to.
- A new pixel value from 0 to 7 is assigned depending on the range the original intensity falls into.
- The quantized image is saved and loaded as a NumPy array for further processing or analysis.

This approach is a basic example of **uniform quantization**, where the intensity range (0–255) is equally divided and mapped to fewer levels.

Q.1. What is the purpose of using .convert('L') when loading the image?

A. To convert the image to RGB
B. To resize the image
C. To convert the image to grayscale
D. To apply a filter

The `.convert('L')` method converts an image to luminance (grayscale) mode. This removes color information and represents the image using only brightness values from 0-255.

Q.2. What type of quantization is implemented in the code?

A. Non-uniform quantization
B. Logarithmic quantization
C. Uniform quantization
D. Variable-level quantization

The code divides the 256 grayscale levels into equal bins (e.g., 0-63, 64-127, etc.). Each pixel value is mapped to a fixed output level based on which bin it falls into, creating uniform steps.

Q.3. How many quantization levels are created in the program?

A. 255
B. 8
C. 16
D. 4

The code uses `quant_levels = 8`, creating 8 distinct output levels.

The 256 grayscale values are divided into 8 equal bins of 32 values each (256 ÷ 8 = 32).

Q.4. Which intensity range is mapped to the quantized value 3?

A. 0–31
B. 64–95
C. 96–127
D. 160–191

With 8 levels (0-7), each bin covers 32 intensity values (256 ÷ 8 = 32).
Level 3 covers: 3 × 32 = 96 to (4 × 32) - 1 = 127.

Q.5. What is the purpose of the pixelMap = im.load() line?

A. It saves the image to disk
B. It loads pixel data into a NumPy array
C. It opens the image file
D. It gives access to individual pixels of the image

`im.load()` returns a PixelAccess object that allows reading and modifying individual pixels.
This enables direct pixel-level manipulation for the quantization process.

Q.6. Which line ensures that the output image remains in grayscale mode?

A. img = Image.new(im.mode, im.size)
B. img = Image.new('L', im.size)
C. im = Image.open('images.jpg')
D. J = numpy.asarray(Image.open('lena_2.jpg'))

The `'L'` parameter explicitly creates a new image in grayscale mode.
This ensures the quantized output maintains the same single-channel grayscale format as the input.

Q.7. What is the role of the line img.save('lena_2.jpg')?

A. It loads a new image
B. It displays the image
C. It writes the quantized image to a file
D. It converts the image to a NumPy array

The `.save()` method stores the modified image data to the specified filename.
This preserves the quantization result as a new JPEG file called 'lena_2.jpg'.

@Exam Buddy

Q.8. Which of the following quantized values corresponds to the intensity range 192 –223?

A. 5
B. 6
C. 4
D. 2

With 8 quantization levels (0-7), each bin covers 32 intensity values.
Level 6 covers: 6 × 32 = 192 to (7 × 32) - 1 = 223.

Q.9. Why is there no separate variable like intensity used in the loop?

A. Because pixelMap[i, j] directly gives the quantized value
B. To save memory by avoiding additional variables
C. To increase the range of pixel values
D. To ensure the image remains in color mode

The code directly calculates and assigns the quantized value to `pixelMap[i, j]` without storing the original pixel intensity in a separate variable.
This optimization avoids creating millions of temporary intensity variables (one per pixel), saving memory during processing.

Q.10. What would happen if the input image was not converted to grayscale before quantization?

A. The program would raise an error
B. The pixelMap[i, j] would return a tuple instead of a single value
C. The quantization would still work correctly
D. The image would be blurred

An RGB image would return a tuple (R, G, B) for each pixel instead of a single grayscale value.
The quantization calculation `pixelMap[i, j] // bin_size` would fail with a TypeError since you can't perform integer division on a tuple.

@Exam Buddy

Imagine you are helping a teacher automate the task of managing student names. The teacher has stored all the names of students who registered for a workshop in a text file called students.txt. Each line in the file contains one student's name.

You've been asked to:

1. Read all the student names from the file.
2. Count how many students registered.
3. Write all the names into a new file registered.txt, prefixed by a serial number (like 1. Riya, 2. Arjun, etc.)

Here is the Python program that does this:

```python
# Step 1: Open the file in read mode
file = open("students.txt", "r")

# Step 2: Read all lines from the file
lines = file.readlines()

# Step 3: Close the file after reading
file.close()

# Step 4: Count how many students registered
count = len(lines)
print("Total students registered:", count)

# Step 5: Open a new file to write the numbered list
new_file = open("registered.txt", "w")

# Step 6: Write each name with a serial number
for i in range(count):
    name = lines[i].strip()  # remove \n
    new_file.write(f"{i+1}. {name}\n")

# Step 7: Close the output file
new_file.close()
```

Q.11. What does the mode "r" in the line open("students.txt", "r") mean?

A. Rewrite the file
B. Read the file
C. Remove the file
D. Run the file

The mode "r" stands for "read" and opens the file for reading only.
This allows the program to access the file's contents but prevents any modifications to the original file.

Q.12. What will file.readlines() return?

A. A string with the entire file content
B. A dictionary of words
C. A list of lines from the file

D. Only the first line of the file

The `readlines()` method reads all lines in the file and returns them as a list of strings. Each element in the list represents one line from the file, including newline characters.

Q.13. What is the purpose of file.close()?

A. Deletes the file
B. Saves memory and finishes file operations
C. Prints the file
D. Converts the file to uppercase

`file.close()` releases system resources associated with the open file.
It ensures all data is properly written and frees up memory used by the file handle.

@Exam Buddy

Q.14. In the line name = lines[i].strip(), what does .strip() do?

A. Removes only spaces
B. Adds a newline character
C. Removes newline and extra spaces from both ends
D. Converts the name to lowercase

The `.strip()` method removes whitespace characters (spaces, tabs, newlines) from both the start and end of a string.
This cleans up the lines read from the file, which typically end with newline characters.

Q.15. What is the effect of opening a file in "w" mode?

A. It appends content to the file
B. It reads content from the file

C. It overwrites the file if it already exists
D. It deletes the file

Opening a file in "w" (write) mode creates a new file or truncates an existing one to zero length.
All previous content is erased before new data is written to the file.

Q.16. What will be written in registered.txt if students.txt contains 3 names?

A. Just the count of students
B. The names with numbering like 1. Name
C. Only the last student
D. Nothing, since the file is empty initially

The code reads each name from students.txt and writes them to registered.txt with numbering.
The format will be: "1. Name1", "2. Name2", "3. Name3" on separate lines.

Q.17. What is the role of \n in the line:

new_file.write(f"{i+1}. {name}\n")?

A. It creates an error
B. It writes a space
C. It moves to a new line after each entry
D. It removes the last line

The \n character represents a newline in text files.
It ensures each numbered student entry appears on a separate line in the output file.

Q.18. Which line in the code ensures that the new file is actually created?

A. open("students.txt", "r")
B. lines = file.readlines()
C. new_file = open("registered.txt", "w")
D. print(count)

Using `"w"` mode automatically creates the file if it doesn't exist.
Even if the file exists, this line ensures a new file is created (or overwritten) for writing.

Q.19. What will len(lines) return?

A. The number of characters in the file
B. The number of lines (students) in the file
C. The size of the file in bytes
D. The first line of the file

`lines` is the list returned by `file.readlines()`, containing all lines from the file.
`len(lines)` counts how many elements (lines/students) are in this list.

Q.20. Why do we use f"{i+1}. {name}\n" instead of i + name?

A. Because numbers and strings cannot be added directly
B. Because i is a float
C. Because we need to store binary values
D. Because name is an integer

Python doesn't allow concatenating integers and strings with the `+` operator.
The f-string format properly converts `i+1` to string and combines it with the name and numbering format.

Week 11 Assignment

Many repetitive online tasks can be automated using Python's **Selenium** library.
For example, logging into a website daily to check a dashboard can be automated with just a few steps.
First, install Selenium by running:

```
pip install selenium
```

Next, download the **Chrome WebDriver** that matches your version of Chrome.
To start writing the script, import the necessary modules:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
import time
```

Launch the Chrome browser:

```
driver = webdriver.Chrome()
```

Open the target website:

```
driver.get("https://www.sales-dashboard.com")
```

Suppose the login form has two input fields:

- Username with id="username"
- Password with id="password"

Enter the credentials:

```
driver.find_element(By.ID, "username").send_keys("user123")
driver.find_element(By.ID, "password").send_keys("mypassword")
```

Submit the form by simulating the **Enter** key:

```
driver.find_element(By.ID, "password").send_keys(Keys.ENTER)
```

Since the page may take time to load, pause execution briefly:

```
time.sleep(5)
```

Finally, close the browser:

```
driver.quit()
```

By following these steps, a Python script can automatically log into a website, eliminating the need for manual browser interaction.

@Exam Buddy

Q.1. Which Python library is used in the passage to automate browser tasks?

A. Selenium
B. BeautifulSoup
C. Requests
D. Pandas

Selenium is specifically designed for automating web browsers and interacting with web elements, making it the correct choice for browser automation tasks mentioned in the passage.

Q.2. Which command installs the Selenium library?

A. install selenium
B. pip install selenium
C. python -m install selenium
D. pip get selenium

This is the standard Python package manager command for installing libraries.
`pip` is Python's package installer that downloads and installs Selenium from the Python Package Index (PyPI).

Q.3. What is the purpose of webdriver.Chrome() in the code?

A. To open a Firefox browser
B. To install Chrome
C. To launch a Chrome browser instance
D. To search Google

`webdriver.Chrome()` initializes and opens a new Chrome browser window.
This creates a ChromeDriver instance that allows programmatic control of the Chrome browser for automation tasks.

Q.4. Which Selenium method opens a given URL in the browser?

A. open()
B. driver.load()
C. driver.goto()
D. driver.get()

The `driver.get(url)` method navigates the browser to the specified URL.
This is Selenium's standard method for loading web pages during browser automation.

Q.5. In the example, what is the id of the username input field?

A. name
B. user
C. username
D. user_id

Based on the typical examples in Selenium automation passages, the username input field commonly uses `id="username"`.
This ID would be used with `driver.find_element(By.ID, "username")` to locate the username field.

Q.6. Which module provides the By class in Selenium?

A. selenium.webdriver.common.by
B. selenium.by
C. webdriver.by
D. selenium.common.by

The `By` class is imported from `selenium.webdriver.common.by`.
It provides constants like `By.ID`, `By.NAME`, `By.XPATH` for locating elements in Selenium.

@Exam Buddy

Q.7. What does send_keys(Keys.ENTER) simulate?

A. Typing text
B. Pressing the Enter key
C. Clicking a button
D. Submitting by mouse

`send_keys(Keys.ENTER)` simulates pressing the Enter key on the keyboard.
This is commonly used to submit forms or trigger actions without needing to locate and click a specific button.

Q.8. Why is time.sleep(5) used in the script?

A. To delay closing the browser until the page loads
B. To speed up execution
C. To refresh the page

D. To avoid importing libraries

`time.sleep(5)` pauses script execution for 5 seconds, allowing time for page elements to load.
This ensures the browser doesn't close before the page fully renders and automation steps can complete.

Q.9. Which Selenium method closes the entire browser window?

A. driver.close()
B. driver.quit()
C. browser.exit()
D. webdriver.stop()

`driver.quit()` closes all browser windows and ends the WebDriver session completely.
This properly cleans up resources and terminates the browser process, unlike `driver.close()` which only closes the current window.

Q.10. What is the main advantage of automating logins with Selenium?

A. It saves manual effort
B. It prevents internet outages
C. It increases battery life
D. It blocks ads

Automating logins with Selenium eliminates the need for manually entering credentials repeatedly.
This saves significant time and effort, especially for testing or routine tasks requiring frequent authentication.

Python provides powerful tools for handling dates, times, and calendar-related tasks through the **datetime** and **calendar** modules.

1. The datetime Module

The datetime module contains several classes, such as:

- **date** – Represents a calendar date (year, month, day).
- **time** – Represents a time of day (hours, minutes, seconds, microseconds).
- **datetime** – Combines both date and time into one object.
- **timedelta** – Represents the difference between two dates or times.

To get the current date and time:

```
import datetime
now = datetime.datetime.now()
```

his returns an object containing year, month, day, hour, minute, and second.
To create a specific date:

```
d = datetime.date(2025, 8, 10)
```

You can extract components:

```
now.year, now.month, now.day
```

You can also format dates using strftime:
Here:

- %A → Full weekday name
- %d → Day of month (zero-padded)
- %B → Full month name
- %Y → Year with century

2. The calendar Module

The calendar module is useful for working with entire months or years. Example: Printing a month's calendar:

```
import calendar
print(calendar.month(2025, 8))
```

Checking if a year is a leap year:

```
calendar.isleap(2024)  # Returns True
```

Getting the weekday of a given date:

```
calendar.weekday(2025, 8, 10)
```

Q.11. Which function from datetime.date returns today's date?

A. date.current()
B. date.now()
C. date.today()
D. date.localtime()

`datetime.date.today()` returns a date object representing the current local date. This is the standard method for getting today's date without time information in Python's datetime module.

@Exam Buddy

Q.12. What does %B stand for in date formatting?

A. Abbreviated month name
B. Full weekday name
C. Full month name
D. Day of the year

In Python's strftime formatting, %B represents the full month name (e.g., "January", "February").
This provides the complete month name rather than the abbreviated version (%b).

Q.13. Which import statement allows direct use of date, time, and datetime without module prefix?

A. import datetime
B. from datetime import module
C. from datetime import all
D. from datetime import date, time, datetime

This import statement brings the specific classes directly into the current namespace. This allows using `date.today()` instead of `datetime.date.today()` for cleaner code.

Q.14. What type does calendar.isleap() return?

A. Integer
B. String
C. Boolean
D. Float

`calendar.isleap(year)` returns either `True` or `False`.
This boolean indicates whether the given year is a leap year according to the Gregorian calendar rules.

Q.15. Which symbol in strftime gives a zero-padded day of the month?

A. %B
B. %A
C. %d
D. %m

In strftime formatting, `%d` represents the zero-padded day of the month (01-31).
This ensures single-digit days appear as two digits (e.g., "05" instead of "5").

Q.16. If calendar.weekday(2023, 12, 31) returns 6, what day is that?

A. Saturday
B. Monday
C. Friday

D. Sunday

`calendar.weekday()` returns integers where Monday=0 and Sunday=6.
A return value of 6 corresponds to Sunday in Python's calendar module.

Q.17. What will datetime.timedelta(days=10) create?

A. A future date 10 days later
B. A calendar for 10 months
C. A list of 10 dates
D. A time duration of 10 days

`datetime.timedelta(days=10)` creates a time duration object representing 10 days.
This can be used for date arithmetic, like calculating dates 10 days in the future or past.

@Exam Buddy

Q.18. Which strftime code outputs a 24-hour format hour?

A. %I
B. %H
C. %p
D. %h

In strftime formatting, `%H` represents the hour in 24-hour format (00-23).
This provides the hour without AM/PM designation, unlike `%I` which gives 12-hour format.

Q.19. What will datetime.date.today().weekday() return if today is Friday?

A. 6
B. 1
C. 4
D. 5

`weekday()` returns integers where Monday=0 through Sunday=6.
Friday is the fifth day of the week (Monday=0, Tuesday=1, Wednesday=2, Thursday=3, Friday=4).

Q.20. Which module provides functions like isleap() and month()?

A. calendar
B. datetime
C. time
D. math

The `calendar` module provides `isleap()` for leap year checks and `month()` for generating monthly calendars.
These are specific functions for calendar-related operations in Python's standard library.

@Exam Buddy

Week 12 Assignment

In the lectures, we discussed a Python program that simulates a simplified version of the PageRank algorithm using **point distribution**. The program creates a directed graph, assigns points to nodes, redistributes those points according to outgoing edges, and compares the results with NetworkX's built-in PageRank function.

1. Graph Creation
A directed graph (DiGraph) with 10 nodes (numbered 0 to 9) is created using NetworkX.

2. Random Edge Generation
The function responsible for adding edges loops over all pairs of distinct nodes (s, t) and adds a directed edge from s to t with a **50% probability**. Self-loops are avoided by checking that s != t.

3. Initial Point Assignment
Every node starts with **100 points**. These points are stored in a list where the index corresponds to the node number.

4. Point Distribution Logic
During redistribution:

- If a node has **no outgoing edges,** it keeps all its points.
- If a node has outgoing edges, its points are **split equally** among them.

5. Iterative Redistribution
The program repeatedly redistributes points over multiple rounds. After each round, the new distribution is printed. The user can stop the process by typing #.

6. Graph Visualization
The graph structure, including nodes and edges, is displayed using matplotlib.

7. Ranking Nodes by Points
At the end, a ranking function stores final points in a dictionary {node: points} and sorts them in ascending order to find which nodes have the lowest and highest scores.

8. Comparison with PageRank
The program also calls NetworkX's built-in pagerank() function to compute the official PageRank values for the same graph and prints them for comparison with the manual simulation.

This simulation helps visualize how **importance flows through a network** and how PageRank emerges as a stable distribution of scores when point redistribution is repeated many times.

Q.1. What type of graph is created in this program?

A. Weighted graph
B. Bipartite graph

C. Directed graph
D. Undirected graph

The passage explicitly mentions creating a "directed graph (DiGraph)" using NetworkX. Directed graphs have edges with direction, unlike undirected graphs where connections are bidirectional.

Q.2. How many nodes are initially added to the graph?

A. 9
B. Random number
C. 10
D. 5

The passage states: "A directed graph (DiGraph) with 10 nodes (numbered 0 to 9) is created using NetworkX."
This specifies exactly 10 nodes are initially added to the graph.

@Exam Buddy

Q.3. Which library is used to handle the graph?

A. numpy
B. matplotlib
C. networkx
D. pandas

The passage repeatedly mentions using NetworkX for graph creation and operations. NetworkX is Python's primary library for creating, manipulating, and studying complex networks and graphs.

Q.4. In add_edges(), what is the probability that an edge will be added between two distinct nodes?

A. 75%
B. 0%
C. 25%
D. 50%

The passage states: "adds a directed edge from s to t with a 50% probability."
This means each potential edge between distinct nodes has a 50% chance of being created.

Q.5. Which condition in add_edges() ensures no self-loop is added?

A. s != t
B. random.random()
C. s == t
D. s > t

The passage explains: "Self-loops are avoided by checking that s != t."
This condition ensures a node doesn't connect to itself, preventing self-loop edges in the graph.

@Exam Buddy

Q.6. Which function assigns the starting points to nodes?

A. pagerank()
B. rank_by_points()
C. add_edges()
D. assign_points()

The passage states: "These points are stored in a list where the index corresponds to the node number."
This describes the `assign_points()` function that initializes each node with 100 points.

Q.7. How many points does each node start with?

A. 50
B. 10
C. 500
D. 100

The passage clearly states: "Every node starts with 100 points."
This initial point assignment is the starting value before any redistribution occurs.

Q.8. If a node has no outgoing edges during distribution, what happens to its points?

A. They remain with the same node
B. They are doubled
C. They are removed
D. They are given to random nodes

The passage explains: "If a node has no outgoing edges, it keeps all its points."
Nodes without outgoing connections don't redistribute their points to other nodes.

@Exam Buddy

Q.9. How are points distributed if a node has multiple outgoing edges?

A. Given entirely to the first edge
B. Divided equally among outgoing edges
C. Distributed randomly
D. Not distributed

The passage states: "If a node has outgoing edges, its points are split equally among them."
This means points are divided uniformly across all outgoing connections from a node.

Q.10. Which function handles the repeated redistribution of points?

A. rank_by_points()
B. keep_distributing()
C. pagerank()
D. add_edges()

The passage mentions: "The program repeatedly redistributes points over multiple rounds."
This describes the `keep_distributing()` function that manages the iterative redistribution process.

Q.11. How does the user stop the redistribution process?

A. Type exit
B. Close the program
C. Press #
D. Press Enter

@Exam Buddy

The passage explicitly states: "The user can stop the process by typing #."
This provides a specific keyboard input (#) to terminate the redistribution loop.

Q.12. What does nx.draw(G, with_labels=True) do?

A. Assigns weights to edges
B. Runs PageRank
C. Adds nodes to the graph
D. Displays the graph visually

The passage states: "The graph structure, including nodes and edges, is displayed using matplotlib."
`nx.draw()` with `with_labels=True` creates a visual representation of the graph with node labels.

Q.13. What data structure does assign_points() return?

A. Dictionary
B. List
C. Tuple
D. Set

The passage explicitly states: "These points are stored in a list where the index corresponds to the node number."
This confirms `assign_points()` returns a list with points indexed by node number.

Q.14. Which function ranks nodes by their final point values?

A. rank_by_points()
B. keep_distributing()
C. pagerank()
D. assign_points()

@Exam Buddy

The passage describes: "At the end, a ranking function stores final points in a dictionary {node: points} and sorts them."
This clearly identifies `rank_by_points()` as the ranking function.

Q.15. What is stored in the dictionary inside rank_by_points()?

A. {node: points}
B. {points: node}
C. {rank: pagerank}
D. {edge: weight}

The passage explicitly states: "a ranking function stores final points in a dictionary {node: points}"
This shows the dictionary maps node identifiers to their point values.

Q.16. Which built-in function is used to calculate actual PageRank values?

A. nx.page_rank()
B. nx.rank()
C. nx.pagerank()
D. nx.graph_rank()

The passage states: "The program also calls NetworkX's built-in pagerank() function"
This is NetworkX's standard function for computing PageRank values on graphs.

Q.17. What is the purpose of comparing manual distribution with nx.pagerank()?

A. To visualize the graph
B. To increase node count
C. To verify the manual simulation
D. To remove edges without points

The passage explains this comparison helps "visualize how importance flows through a network and how PageRank emerges."
This verification ensures the manual point redistribution correctly simulates the PageRank algorithm.

Q.18. Which library is responsible for random edge creation?

A. random
B. itertools
C. math
D. numpy

The passage mentions using "random edge generation" and "50% probability" for edge creation.
The `random` library provides the `random()` function used for probabilistic edge

generation.

Q.19. If random.random() returns 0.7, will an edge be added between two distinct nodes?

A. Yes
B. No
C. Only if s > t
D. Only if s < t

The passage states edges are added with "50% probability," meaning when `random.random() < 0.5`.
Since 0.7 > 0.5, no edge will be added between the distinct nodes.

Q.20. What is the total number of points in the system at the start?

A. 1000
B. Depends on number of edges
C. 500
D. 100

With 10 nodes each starting with 100 points, the total is 10 × 100 = 1000 points.
The passage confirms this initial point total before redistribution begins.